



République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
École Supérieure Privée d'ingénierie et de technologie
TEK-UP



RAPPORT DE PROJET DE FIN D'ÉTUDES

Présenté en vue de l'obtention du
Diplôme National d'Ingénieur en Sciences Appliquées et Technologiques
Spécialité : Filière

Réalisé par

XXXXXXXXXXXXXXXXXX

Conception et Développement d'une Plateforme Centralisée de Gestion et de Pilotage Financier des Projets Informatiques (PMS)

Encadrant professionnel : **Monsieur Prénom NOM**

Ingénieur Informatique

Encadrant académique : **Monsieur ZZZZZZZ**

Poste

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant professionnel, **M. XXXXXXXX**

Signature et cachet

J'autorise l'étudiant à faire le dépôt de son rapport de stage en vue d'une soutenance.

Encadrant académique, **M. YYYYYYY**

Signature

Dédicace

Remerciements

Table des matières

General Introduction	1
1 Project Context and Methodology	3
1.1 Project Framework	4
1.2 Host Organization	4
1.3 Study of the Existing System	4
1.3.1 The Current Process	4
1.3.2 Analysis of the Reference Workbook	5
1.3.3 Critique of the Existing System	9
1.4 Problem Statement	9
1.5 Proposed Solution	10
1.6 Development Methodology	10
1.6.1 An Incremental and Iterative Approach	10
1.6.2 Engineering Phases and Development Sprints	10
1.6.3 Organisation into Releases and Sprints	11
1.6.4 Sources of Truth	11
2 Requirements Analysis and Specification	13
2.1 Actor Identification	14
2.2 Functional Requirements	14
2.3 Non-Functional Requirements	15
2.4 Business Rules	16
2.5 The Dynamic Authorization Model	17
2.6 Global Use Case Diagram	17
2.7 Product Backlog	19
2.8 Release and Sprint Planning	21
3 System Architecture and Design	22
3.1 Architectural Style	23
3.1.1 A Three-Tier Modular Monolith	23
3.1.2 Layered Organisation	24
3.2 Cross-Cutting Design Decisions	25

3.2.1	Authentication and Session Revocation	25
3.2.2	Dynamic Authorization	25
3.2.3	A Hybrid Indicator Engine	25
3.2.4	Multi-Currency Handling	25
3.2.5	Auditability and Logical Deletion	26
3.3	Domain Model	26
3.4	Database Design	26
3.4.1	Modelling Conventions	26
3.4.2	Logical Model	27
3.4.3	Monthly Time Series and Indicator Snapshots	28
3.4.4	Reference Data and Migration Strategy	28
3.5	Deployment Architecture	28
3.6	Technical Environment	29
4	Release 1 — Foundation and Security	30
4.1	Sprint 1 — Technical Foundation and Authentication	31
4.1.1	Sprint Objective	31
4.1.2	Functional Scope	31
4.1.3	Features Implemented	31
4.1.4	Technical Realisation	31
4.1.5	Tests and Validation	33
4.1.6	Sprint Outcome	33
4.1.7	Transition to the Next Sprint	33
4.2	Sprint 2 — Dynamic Access Control, Users and Resources	34
4.2.1	Sprint Objective	34
4.2.2	Functional Scope	34
4.2.3	Features Implemented	34
4.2.4	Technical Realisation	34
4.2.5	Tests and Validation	35
4.2.6	Sprint Outcome	36
4.2.7	Transition to the Next Sprint	36
5	Release 2 — Project and Operational Management	37
5.1	Sprint 3 — Project Management and Lifecycle	38
5.1.1	Sprint Objective	38

5.1.2	Functional Scope	38
5.1.3	Features Implemented	38
5.1.4	Technical Realisation	38
5.1.5	Tests and Validation	40
5.1.6	Sprint Outcome	40
5.1.7	Transition to the Next Sprint	41
5.2	Sprint 4 — Teams and Workload Management	41
5.2.1	Sprint Objective	41
5.2.2	Functional Scope	41
5.2.3	Features Implemented	41
5.2.4	Technical Realisation	41
5.2.5	Tests and Validation	42
5.2.6	Sprint Outcome	43
5.2.7	Transition to the Next Sprint	43
6	Release 3 — Financial Management and Governance	44
6.1	Sprint 5 — Billing and Missions	45
6.1.1	Sprint Objective	45
6.1.2	Functional Scope	45
6.1.3	Features Implemented	45
6.1.4	Technical Realisation	45
6.1.5	Tests and Validation	46
6.1.6	Sprint Outcome	47
6.1.7	Transition to the Next Sprint	47
6.2	Sprint 6 — Project Governance	47
6.2.1	Sprint Objective	47
6.2.2	Functional Scope	47
6.2.3	Features Implemented	47
6.2.4	Technical Realisation	48
6.2.5	Tests and Validation	48
6.2.6	Sprint Outcome	49
6.2.7	Transition to the Next Sprint	49
6.3	Sprint 7 — Internal Quote and Indicator Engine	49
6.3.1	Sprint Objective	49

6.3.2	Functional Scope	49
6.3.3	Features Implemented	49
6.3.4	Technical Realisation	49
6.3.5	Tests and Validation	51
6.3.6	Sprint Outcome	52
6.3.7	Transition to the Next Sprint	52
7	Release 4 — Consolidation, Delivery and Deployment	53
7.1	Sprint 8 — Interface Consolidation, Testing and Deployment	54
7.1.1	Sprint Objective	54
7.1.2	Functional Scope	54
7.1.3	Features Implemented	54
7.1.4	Technical Realisation	54
7.1.5	Tests and Validation	56
7.1.6	Sprint Outcome	57
7.1.7	Transition to the Next Sprint	57
7.2	Sprint 9 — Containerization and Continuous Delivery	57
7.2.1	Sprint Objective	57
7.2.2	Functional Scope	57
7.2.3	Features Implemented	58
7.2.4	Technical Realisation	58
7.2.5	Tests and Validation	59
7.2.6	Sprint Outcome	60
7.2.7	Final State of the Product	60
	General Conclusion and Perspectives	62
	Webographie	64

Table des figures

2.1	Global use case diagram of the PMS platform	18
3.1	Functional modules and their main dependencies	24
3.2	Domain overview — entities grouped by sub-domain	26
3.3	Class diagram — security and access control	26
3.4	Deployment architecture	29
5.1	Class diagram — projects and teams	39
5.2	State diagram — project lifecycle	39
5.3	Sequence — assigning a developer, permission and scope	42
6.1	Class diagram — financial management	46
6.2	Class diagram — project governance	48
6.3	Class diagram — workload and indicator engine	50
6.4	Sequence — workload entry and indicator computation	51

Liste des tableaux

1.1	Functional structure of the Excel project review workbook	5
1.2	Traceability from the Excel workbook to the PMS modules	9
1.3	Organisation of the realisation into releases and sprints	11
2.1	PMS actors and their boundaries	14
2.2	Functional requirements by module	15
2.3	Summary of business rules	17
2.4	Product backlog of the PMS platform	21
2.5	Distribution of the backlog across releases and sprints	21
3.1	Catalogue of entities by domain	28
3.2	Technical environment of the project	29
4.1	Test cases — Sprint 1	33
4.2	Test cases — Sprint 2	36
5.1	Test cases — Sprint 3	40
5.2	Test cases — Sprint 4	43
6.1	Test cases — Sprint 5	47
6.2	Test cases — Sprint 6	48
6.3	Test cases — Sprint 7	52
7.1	Security review by vulnerability category	56
7.2	Test cases — Sprint 8	57
7.3	Test cases — Sprint 9	60
7.4	From the reference workbook to the delivered platform	61

List of Abbreviations

- **API** = Application Programming Interface
- **BR** = Business Rule
- **BRS** = Business Requirements Specification
- **EAC** = Estimate At Completion
- **ETC** = Estimate To Complete
- **EV** = Earned Value
- **FAE** = Facture À Établir — revenue earned but not yet invoiced
- **FCFA** = Franc of the African Financial Community
- **FP** = Fixed Price — contract billed at an agreed total
- **FR** = Functional Requirement
- **HTTP** = Hypertext Transfer Protocol
- **JPA** = Java Persistence API
- **JWT** = JSON Web Token
- **KPI** = Key Performance Indicator
- **NFR** = Non-Functional Requirement
- **ORM** = Object-Relational Mapping
- **PFE** = Projet de Fin d'Études — final-year graduation project
- **PMS** = Project Management System
- **PPR** = Provision Pour Risques — risk provision withheld from margin
- **RBAC** = Role-Based Access Control
- **REST** = Representational State Transfer
- **SQL** = Structured Query Language
- **SRS** = Software Requirements Specification

- **TCC** = Taux de Coût Chargé — fully loaded cost rate of a resource
- **TND** = Tunisian Dinar
- **UC** = Use Case
- **UML** = Unified Modeling Language
- **US** = User Story

General Introduction

In a digital services company, profitability is decided project by project. Each engagement sells a volume of effort at a negotiated price, consumes resources whose cost is known, and is invoiced according to a contractual schedule. Steering therefore means keeping three quantities under control at once : the workload sold, the effort actually consumed, and the revenue invoiced. Losing sight of any one of them means discovering a loss-making project only once it is finished.

The host company steers this equilibrium through an Excel workbook, a “project review sheet” of some twenty interconnected tabs and more than two thousand formulas, duplicated for every project. This workbook is remarkable in what it encodes : loaded cost rates per resource and per year, monthly workload plans, billing milestones as a percentage of the contract, mission expenses, governance registers and a complete earned-value indicator engine. It is, in practice, the company’s real specification.

It is also a fragile medium. One file per project means no single source of truth ; formulas spanning tabs break silently, and the file provided for this project indeed contains cells in error ; consolidating the portfolio is impossible without reopening every workbook ; and a spreadsheet cannot prevent a developer from reading the margin of the project they work on.

This final year project addresses that gap through **PMS** (*Project Management System*), a centralised web platform that automates the business logic carried by the workbook. Two requirements shaped it. The first is fidelity : the platform must reproduce the financial model faithfully (cost rates, workload plan, estimate at completion, margins, billing progress) rather than a simplified approximation of it. The second is configurability : the authorization policy must be modifiable as data, without redeploying the application, so that the platform can absorb the evolution of the company’s rules.

The platform was built incrementally, in nine successive sprints grouped into four releases, each delivering a demonstrable increment of the product. This report follows that construction.

Chapter 1 situates the project : the host organization, the analysis of the existing Excel-based process, including the representative sheets from which each module derives, the resulting problem statement, and the methodology adopted.

Chapter 2 specifies the system : actors and their boundaries, functional and non-functional requirements, business rules, the dynamic authorization model, the global use case diagram, and the product backlog and its distribution across the development sprints.

Chapter 3 presents the technical design : the architectural style and its justification, the cross-cutting design decisions, the domain model and the relational schema.

Chapters 4 to 7 present the realisation, release by release : the foundation and security of Release 1, the operational management of Release 2, the financial and governance value of Release 3, and the consolidation, testing and deployment of Release 4.

A **general conclusion** assesses the result against the initial objectives and outlines the perspectives opened by the platform.

PROJECT CONTEXT AND METHODOLOGY

Plan

1	Project Framework	4
2	Host Organization	4
3	Study of the Existing System	4
4	Problem Statement	9
5	Proposed Solution	10
6	Development Methodology	10

Introduction

This first chapter situates the project in its context. It presents the host organization, analyses the existing project-tracking process, built on Excel workbooks, and identifies its limitations. From this study follow the problem statement, the proposed solution (the PMS platform), and the development methodology adopted to build it.

1.1 Project Framework

This work constitutes a Final Year Project (*Projet de Fin d'Études*) submitted for the National Engineering Diploma. It was carried out within a digital services company (ESN) that designs and implements information systems for its clients under contract, most often on a fixed-price (FP) or time-and-material (T&M) basis.

1.2 Host Organization

The host company operates as a software solutions integrator. Its activity rests on running several projects simultaneously, mobilising a pool of resources (developers, business analysts, project managers) assigned to client projects. The company's profitability depends directly on its ability to control, for each project, the balance between the workload sold, the workload actually consumed, and the revenue invoiced.

1.3 Study of the Existing System

1.3.1 The Current Process

Each project is tracked through an Excel workbook, the “project review sheet”, comprising around twenty interconnected tabs. For a given project, this workbook centralises all steering information : project identification and budget, resource cost rates, planned workload, actual workload, billing milestones, missions, risks, deliverables and dashboards.

Tab family	Business role
Identification	Project identification sheet : client, contract, budget, currency, duration, project manager, workload sold.
Costs (TCC, Forecast Cost, Updated Actual Cost)	Computation of the daily cost of resources and of forecast and actual costs per month.

Tab family	Business role
Billing (Progress, Summary)	Billing milestones as a percentage of the contract, tracking of invoiced and settled amounts.
Missions	Computation of travel costs (per diem, ticket, accommodation).
Steering (Statistics, Dashboard, Glossary)	Consolidation of indicators and KPI definitions.
Governance (Risks, Deliverables, Stakeholders, Changes, Amendments)	Registers tracking the life of the project.

TABLEAU 1.1 : Functional structure of the Excel project review workbook

1.3.2 Analysis of the Reference Workbook

The workbook provided by the company (reference **F-AFF-13**, version 03) is the true functional specification of the platform : it contains twenty-one sheets and more than two thousand formulas that encode the company's real steering logic. Its analysis constituted the preparation phase of this project, and each PMS module derives directly from one or more of these sheets.

The eight most representative sheets are presented below. The figures shown are **anonymised** : client name, contract reference, staff names and the amounts of the internal quote have been replaced by neutral values, as this information is covered by the confidentiality of the host company and of its client. The structure, the formulas and the business logic, the only elements relevant to this analysis, are preserved.

1.3.2.1 Project identification sheet

What it represents. The identity card of the project : client, contract reference, funder, business model (alone or in consortium), engagement type (fixed price or time & material), project manager, start and end dates, contract duration, project currency and initial budget.

Business purpose. It is the entry point of every project review. It fixes the contractual and financial baseline against which all later indicators are measured.

Key information and calculations. The sheet converts the budget from the project currency (here FCFA) into the reporting currency (TND) using a contractual exchange rate ; it computes the risk provision $PPR = \text{Budget} \times 5\%$, the penalty provision (PPP), the total cost of the internal quote and the *sold net margin* $(\text{Selling price} - \text{Total cost}) / \text{Selling price}$. It also carries the amendment fields, which adjust the total budget.

Derived PMS functionality. The *Project management* module : project creation with code, client, funder, engagement type, multi-currency budget, provisions and sold workload ; and the *Internal quote*, which reproduces this economic baseline.

1.3.2.2 Cost rate sheet (TCC)

What it represents. The daily loaded cost (*Taux de Coût Chargé*) of each resource, recorded per year.

Business purpose. It converts a human resource into a unit cost. Every cost computation in the workbook, forecast and actual alike, ultimately multiplies man-days by a value taken from this sheet.

Key information and calculations. The daily rate is derived from the annual loaded cost by $TCC_{jour} = (\text{annual loaded cost} \times 1.7) / \text{working days per year}$, where the overhead coefficient covers employer charges and the number of working days differs from one year to the next (22 in 2024, 20 in 2025 in the reference file). A validated historical value must remain immutable, otherwise past reviews would change retroactively.

Derived PMS functionality. The *Resource and yearly cost rate* module (Sprint 2) : one rate per resource and per year, with the overhead coefficient and the number of working days externalised as configurable business parameters rather than hard-coded constants.

1.3.2.3 Planned workload sheet

What it represents. A resource month matrix giving the man-days planned for each resource over the life of the project, together with the resulting forecast cost.

Business purpose. It is the workload plan : it expresses how the sold effort is expected to be consumed over time, and therefore what the project should cost.

Key information and calculations. With around four hundred formulas, this sheet computes the monthly and cumulative totals of planned man-days, the forecast cost $\Sigma (\text{planned man-days} \times TCC)$, and the *Estimate To Complete* $ETC = \Sigma (\text{remaining man-days} \times TCC)$ as a running remainder.

Derived PMS functionality. The *Workload planning* module (Sprint 4), which stores the plan as a monthly time series per resource, and feeds the ETC used by the KPI engine.

1.3.2.4 Actual workload sheet

What it represents. The mirror of the previous sheet, filled with the effort actually consumed : the man-days really booked by each resource, month by month.

Business purpose. It measures reality against the plan. The gap between the two sheets is

what reveals a project drifting.

Key information and calculations. It is the densest sheet of the workbook (more than five hundred formulas). Effort may be captured in hours and normalised into man-days by **man-days** = **hours** / 8. The actual cost follows the same rule as the forecast, Σ (**consumed man-days** **TCC**), with monthly and cumulative totals. History must be preserved : an entry is never deleted.

Derived PMS functionality. The *Actual workload* module (Sprint 4) : submission restricted to assigned projects, validation by the project manager, preserved history, and the actual cost that feeds the EAC.

1.3.2.5 Billing progress sheet

What it represents. The billing milestones of the contract, expressed as a percentage of the total, with their amount, their status and the monthly cumulative invoiced total.

Business purpose. It tracks the revenue side of the project : what has been invoiced, what has been collected, and what remains to be billed.

Key information and calculations. Each milestone amount is computed as **contract total percentage**; the sheet maintains the invoiced cumulative total, the remaining amount, and the billing progress **invoiced cumulative / contract total**. A structural rule governs the sheet : the sum of milestone percentages may not exceed 100 %.

Derived PMS functionality. The *Billing* module (Sprint 5) : milestones with percentage, amount and lifecycle status, payments recorded against a milestone, and the automatic billing-progress indicator.

1.3.2.6 Missions sheet

What it represents. The business trips carried out for the project, with their cost components : per diem, plane ticket, accommodation, transport and administrative stamp.

Business purpose. Travel costs are a real charge on the project margin ; they must be attached to the project rather than treated as a general overhead.

Key information and calculations. The total cost of a mission is the sum of its components, with the per diem obtained by multiplying the number of nights by a daily allowance converted at the applicable exchange rate.

Derived PMS functionality. The *Missions* module (Sprint 5) : a mission attached to a project and an employee, with multi-currency cost components and an automatically computed total.

1.3.2.7 Project dashboard sheet

What it represents. The steering board of a single project : margins, man-days consumed against sold, key dates and the overall health of the project.

Business purpose. It is the sheet the project manager and the director actually look at during the monthly review. It answers one question : is this project still profitable ?

Key information and calculations. The dashboard does not compute the raw data ; it reads values produced by the other sheets through lookups and presents the margins and progress indicators. This separation between computation and presentation is precisely the design later adopted in PMS.

Derived PMS functionality. The *KPI dashboards* (Sprint 7) : a per-project indicator board, and its portfolio-level counterpart for the director.

1.3.2.8 Statistics sheet — KPI evolution

What it represents. The real computation engine of the workbook : the month-by-month evolution of the financial indicators of the project.

Business purpose. It transforms the raw data of the other sheets into the earned-value indicators used to steer the project and to detect drift early.

Key information and calculations. This is the most formula-dense sheet of the workbook (more than six hundred formulas). It computes, per selected month, the production revenue `contract total` `EV %`, the estimate at completion `EAC = actual cost + ETC`, the net margin at `EAC Budget - EAC`, the accrued income (FAE) as the difference between production revenue and the invoiced total, and the drift in man-days between what was sold, consumed and remaining.

Derived PMS functionality. The *KPI engine* (Sprint 7). This sheet is the direct origin of the hybrid design retained in PMS : indicators recomputed on read from the live data, plus a frozen monthly snapshot that preserves the official state of each review. This snapshot is the digital equivalent of the monthly column of this sheet.

1.3.2.9 Traceability towards the platform

Table 1.2 summarises the correspondence established during this preparation phase between the sheets analysed and the modules built.

Excel sheet	Business logic captured	PMS module (sprint)
Fiche identification	Contractual and financial baseline, provisions, sold margin	Project management (S3), Internal quote (S7)

Excel sheet	Business logic captured	PMS module (sprint)
TCC	Daily loaded cost per resource and per year	Resources and cost rates (S2)
Coût Prévisionnel	Workload plan and forecast cost, ETC	Workload planning (S4)
Coût réel Actualisé	Consumed effort and actual cost	Actual workload (S4)
Avancement_Facturation	Milestones in % of contract, billing progress	Billing (S5)
Missions	Travel cost components	Missions (S5)
Dashboard	Per-project indicator board	KPI dashboards (S7)
Statistiques	Earned-value engine, EAC, margins, accrued income	KPI engine (S7)

TABLEAU 1.2 : Traceability from the Excel workbook to the PMS modules

1.3.3 Critique of the Existing System

While this workbook expresses a proven business logic, its medium, the spreadsheet, introduces structural limitations :

- **Dispersion** : one workbook per project, duplicated and liable to diverge from one version to the next, with no single source of truth.
- **Computation reliability** : the dependency on formulas and cross-tab references is fragile ; the actual file provided contains cells in error (**#REF!**, **#VALUE!**).
- **Financial visibility** : the absence of consolidation makes it difficult to read actual cost and margin at portfolio level.
- **Access control** : a spreadsheet cannot partition information finely ; it cannot, for instance, prevent a developer from accessing financial data.
- **Traceability** : the history of modifications and decisions is not reliably preserved.

1.4 Problem Statement

The central question is therefore the following : *how can the company be equipped with a centralised, reliable and secure tool that automates the project financial logic currently carried by Excel,*

while remaining configurable enough to absorb the evolution of business rules without a rewrite? Two requirements emerge : preserving the richness of the existing business model (cost rates, workload plan, EAC, margins, billing) and guaranteeing a *dynamic* authorization architecture, decoupled from roles.

1.5 Proposed Solution

The chosen solution is the development of a centralised web platform, **PMS** (*Project Management System*). It provides a single source of information and automates the operational and financial steering of projects. Its objectives are to :

- centralise project, resource and team data ;
- plan workload (in man-days) and record actual workload ;
- manage billing milestones and missions ;
- automatically compute financial indicators (EAC, margins, billing progress, remaining work, drift) ;
- provide decision-support dashboards tailored to each user profile ;
- rest on a dynamic “Role → Permission → Module” access control.

1.6 Development Methodology

1.6.1 An Incremental and Iterative Approach

The construction of PMS follows an **incremental and iterative** approach. Rather than delivering the whole platform at once, the work was broken down into successive increments, each one delivering a coherent and demonstrable subset of the product : a functional perimeter, its persistence, its REST API, its access rules and its user interface.

This choice is dictated by the nature of the domain. The PMS business model is deeply interdependent : the KPI engine consumes workload, billing, cost rates and the internal quote. It cannot be built in a single pass without the risk of discovering integration errors too late. Proceeding by increments allows each functional block to be validated against the reference Excel model before the next one is built on top of it.

1.6.2 Engineering Phases and Development Sprints

Two distinct notions structure this report and must not be confused :

- The **engineering phases** (business analysis, architecture design, database design and UML modelling) constitute the upstream study work. They are presented in Chapters 2 and 3.
- The **development sprints** describe the progressive construction of the software. Each sprint delivers a functional increment of the platform. They are presented in Chapters 4 to 7.

1.6.3 Organisation into Releases and Sprints

The realisation was organised into **four releases** grouping **nine sprints**. Each release corresponds to a coherent stage of product maturity, and each sprint to an increment that can be demonstrated in isolation.

Release	Sprint	Functional increment delivered
R1 — Foundation	Sprint 1	Technical foundation and authentication
	Sprint 2	Dynamic RBAC, users and resources
R2 — Operations	Sprint 3	Project management and lifecycle
	Sprint 4	Teams and workload management
R3 — Finance	Sprint 5	Billing and missions
	Sprint 6	Project governance
	Sprint 7	Internal quote and KPI engine
R4 — Delivery	Sprint 8	Consolidation, testing and deployment

TABLEAU 1.3 : Organisation of the realisation into releases and sprints

Each sprint is presented in this report according to a constant structure : objective, functional scope, features implemented, technical realisation, tests and validation, sprint outcome, and transition to the next sprint.

1.6.4 Sources of Truth

Project conduct also relies on a **priority of sources of truth** : validated architecture decisions take precedence, followed by business instructions, then the reality extracted from the Excel file, then the specifications (BRS, use cases, SRS), and finally the project plan, considered a strategic orientation to be challenged.

Conclusion

The study of the context and of the existing system highlighted the need to replace scattered Excel tracking with a centralised, reliable and configurable platform, and set out the incremental

approach adopted to build it. The next chapter deepens this ambition by analysing and specifying precisely the functional and non-functional requirements of the system, together with the product backlog that structures the development sprints.

REQUIREMENTS ANALYSIS AND SPECIFICATION

Plan

1	Actor Identification	14
2	Functional Requirements	14
3	Non-Functional Requirements	15
4	Business Rules	16
5	The Dynamic Authorization Model	17
6	Global Use Case Diagram	17
7	Product Backlog	19
8	Release and Sprint Planning	21

Introduction

This chapter formalises the business analysis of the PMS system. It identifies the actors, lists the functional and non-functional requirements, presents the business rules and the dynamic authorization model that structures the whole platform, and derives from them the global use case diagram. It closes with the product backlog and its distribution across the development sprints presented in the following chapters.

2.1 Actor Identification

The system rests on four main roles. Each role is defined as much by what it may do as by what it is explicitly prevented from doing, a distinction that is central to a platform handling sensitive financial data.

Actor	Mission	Boundary (cannot)
Administrator	Technical and functional administration : users, roles, permissions, cost rates, configuration.	Steer projects ; access project financial data.
Director	Strategic governance : project creation, budgets, project manager assignment, portfolio supervision, executive KPIs.	Manage day-to-day operations (delegated, not forbidden).
Project Manager	Main operational user : teams, workload plan, billing, missions, cost and profitability tracking.	Manage users, roles, cost rates and configuration.
Developer	Entry of actual workload ; consultation of own assignments and missions.	Access any financial data (budget, margin, EAC, billing).

TABLEAU 2.1 : PMS actors and their boundaries

2.2 Functional Requirements

Functional requirements are organised by module. They synthesise the requirements of the SRS document (FR-001 to FR-053) and the use cases (UC-001 to UC-020).

Module	Main functional requirements
--------	------------------------------

Module	Main functional requirements
Authentication	Login by e-mail and password, generation of JWT access and refresh tokens, mandatory password change on first login, logout.
User management	Creation by the administrator, automatic generation of a temporary password, modification, activation and deactivation, role assignment.
Roles and permissions	Management of roles, permissions and role-permission associations.
Project management	Creation, modification, status change, budget definition, project manager assignment, portfolio consultation.
Team management	Assignment and removal of developers, availability consultation, assignment history.
Cost rate (TCC) management	Entry of the loaded cost per resource and per year, history, daily cost computation.
Workload plan	Monthly planning in man-days, planned totals, ETC and forecast computation.
Actual workload	Entry of man-days consumed (assigned projects only), validation, history.
Billing	Milestone creation (percentage, amount, dates), status tracking, progress computation.
Missions	Mission creation, expense entry, automatic total cost computation.
KPI and reporting	Financial indicator computation, project and portfolio dashboards, report generation.
Governance (amendments, risks, deliverables, stakeholders, changes)	Registers tracking the life of the project, in line with the Excel workbook.

TABLEAU 2.2 : Functional requirements by module

2.3 Non-Functional Requirements

- **Performance** : response in under two seconds for routine operations (NFR-001).

- **Security** : mandatory JWT authentication, hashed passwords (NFR-002).
- **Authorization** : permission-based access control (dynamic RBAC, NFR-003).
- **Maintainability** : clean, modular, feature-oriented architecture (NFR-006).
- **Scalability** : support for growth and future integration (ERP, HR, accounting) without rewriting the core (NFR-005, BR-065).
- **Auditability** : history preservation and logical deletion (NFR-008).
- **Usability** : intuitive and responsive interface (NFR-007).

2.4 Business Rules

The business rules come from the BRS document (65 rules, BR-001 to BR-065), enriched by the rules extracted from the Excel file. The most structuring ones are summarised below.

Domain	Main rules
Users	Creation reserved to the administrator; no self-registration; a single role per user; unique e-mail; temporary password then change on first login.
Projects	Creation reserved to the director; exactly one director and one active project manager per project; status among {DRAFT, ACTIVE, ON_HOLD, COMPLETED, CANCELLED}; unique project code; strictly positive budget; start date before end date.
Teams	A developer may belong to several projects; removal preserves history; an assignment records its author and date; inactive developers cannot be assigned.
Cost rates	Managed by the administrator; one value per resource and per year; historical value immutable after financial validation.
Workload	Workload expressed in man-days, never negative; entry restricted to assigned projects; history preserved (no deletion).
Billing	A milestone belongs to one project; status among {PENDING, INVOICED, PAID}; positive amount; the sum of milestone percentages is at most 100 %.

Domain	Main rules
Missions	A mission belongs to a project and an employee ; start date before end date ; total cost computed automatically.
KPI and security	Financial KPIs visible only to the director and the project manager ; automatic computation ; RBAC applied by permission ; hashed passwords.

TABLEAU 2.3 : Summary of business rules

2.5 The Dynamic Authorization Model

The central architectural requirement of the project is to **never couple a role directly to a module**. Such a coupling (“ROLE → MODULE”) would require modifying and redeploying the code every time the company decides that a given profile may access an additional function.

The model retained therefore inserts an intermediate level :

$$\mathbf{Role} \rightarrow \mathbf{Permission} \rightarrow \mathbf{Module}$$

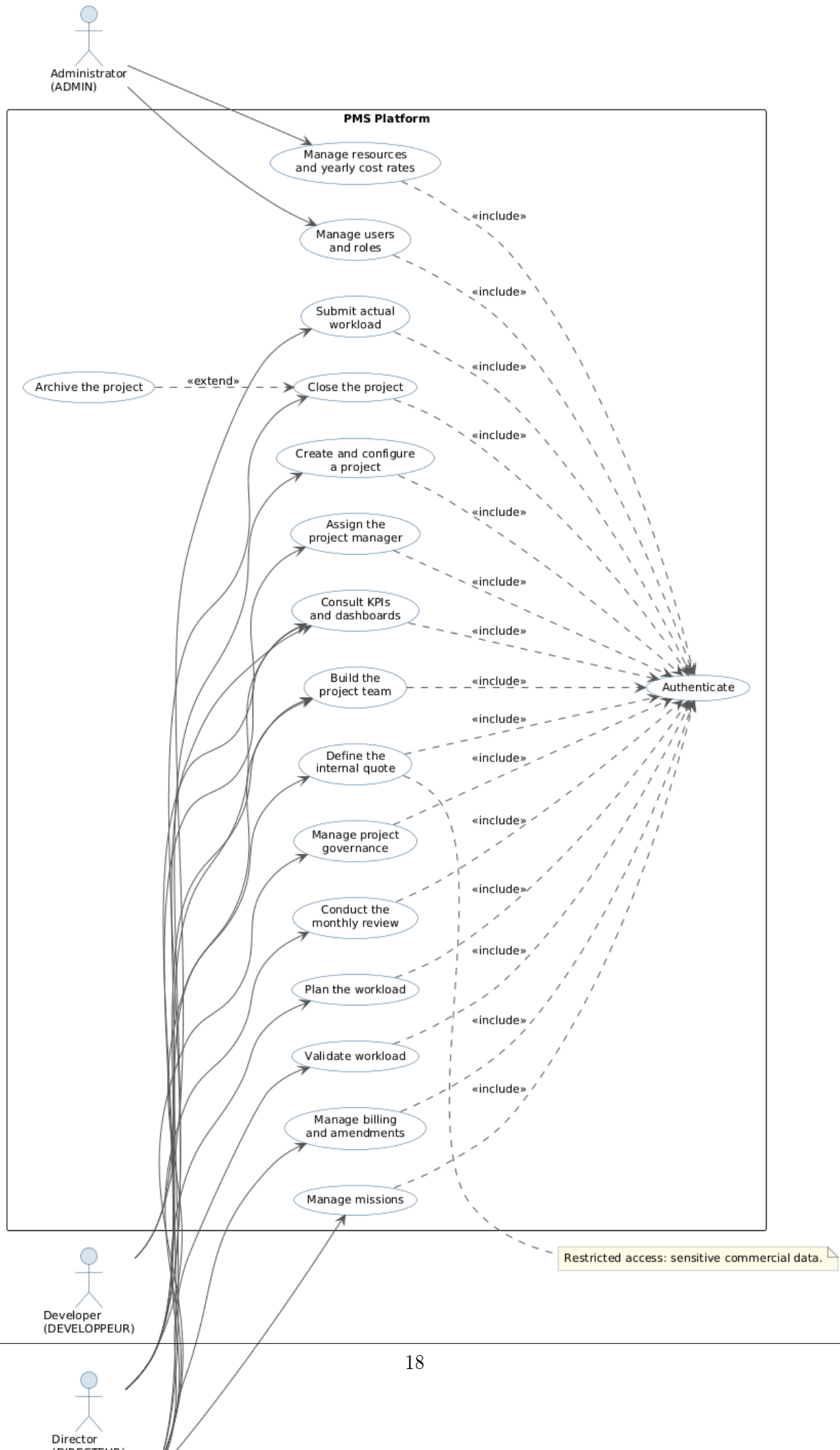
A role holds a set of permissions ; each permission authorises a precise operation on a module. The role–permission matrix lives in the database, not in the source code : granting a new right becomes a data change, applied without redeployment. This principle governs the whole platform and is verified in Sprint 2 (Chapter 4).

2.6 Global Use Case Diagram

Figure 2.1 presents the global use case diagram of the platform. The four actors are associated with the business use cases falling within their perimeter, in strict accordance with the boundaries stated in Table 2.1 : the administrator holds no project use case, and the developer holds no financial use case.

Any functionality requiring a prior identification points to the “Authenticate” use case through an *include* relation, which avoids repeating the same association on each actor. The “Archive the project” use case, optional after closure, extends “Close the project” through an *extend* relation.

Global Use Case Diagram — PMS Platform



2.7 Product Backlog

The product backlog gathers the functional needs expressed as user stories. Each item is attached to the sprint that delivered it, which makes the incremental construction of the platform legible. Priority is derived from technical dependency : a story is high priority when subsequent increments depend on it.

Id	User story	Priority	Sprint
US-01	As a user, I want to sign in with my e-mail and password so that I can access the platform.	High	Sprint 1
US-02	As a user, I want to be forced to change my password on first login so that my account is secured.	High	Sprint 1
US-03	As an administrator, I want a user's sessions to be revoked immediately when needed so that access can be cut off without waiting for token expiry.	High	Sprint 1
US-04	As an administrator, I want to assign permissions to roles so that authorization evolves without redeploying the application.	High	Sprint 2
US-05	As a user, I want the menu to show only what I am entitled to so that the interface matches my rights.	High	Sprint 2
US-06	As an administrator, I want to create, modify and deactivate user accounts so that I can manage platform access.	High	Sprint 2
US-07	As an administrator, I want to record resources and their yearly cost rate so that project costs can be computed.	High	Sprint 2
US-08	As a director, I want to create and configure a project so that it can be tracked on the platform.	High	Sprint 3
US-09	As a director, I want a project to follow a controlled lifecycle so that no invalid status transition is possible.	Medium	Sprint 3
US-10	As a director, I want to assign a project manager to a project so that its operational steering is delegated.	High	Sprint 3
US-11	As a project manager, I want to build my project team so that I can allocate developers to it.	High	Sprint 4

Id	User story	Priority	Sprint
US-12	As a project manager, I want to plan the monthly workload so that I can forecast the effort required.	High	Sprint 4
US-13	As a developer, I want to submit the man-days I worked so that actual consumption is tracked.	High	Sprint 4
US-14	As a project manager, I want to validate the workload submitted so that only verified data feeds the indicators.	Medium	Sprint 4
US-15	As a project manager, I want to define billing milestones so that invoicing follows contractual progress.	High	Sprint 5
US-16	As a project manager, I want to record payments received against a milestone so that collection is tracked.	Medium	Sprint 5
US-17	As a project manager, I want to record amendments so that the revised budget reflects contractual changes.	Medium	Sprint 5
US-18	As a project manager, I want to record missions and their expenses so that travel costs are included in the project cost.	Medium	Sprint 5
US-19	As a project manager, I want to keep a risk register so that project risks are tracked and mitigated.	Medium	Sprint 6
US-20	As a project manager, I want to track deliverables and their status so that delivery progress is measurable.	Medium	Sprint 6
US-21	As a project manager, I want to record stakeholders and change requests so that project governance is documented.	Low	Sprint 6
US-22	As a director, I want to define the internal quote so that the project has an economic baseline.	High	Sprint 7
US-23	As a project manager, I want the platform to compute the financial indicators automatically so that I no longer depend on the Excel workbook.	High	Sprint 7
US-24	As a project manager, I want to freeze a monthly review so that the official state of the project is kept as history.	Medium	Sprint 7
US-25	As a director, I want a portfolio dashboard so that I can supervise all projects at a glance.	High	Sprint 7

Id	User story	Priority	Sprint
US-26	As a user, I want a consistent and responsive interface so that the platform is comfortable to use daily.	Medium	Sprint 8

TABLEAU 2.4 : Product backlog of the PMS platform

2.8 Release and Sprint Planning

The backlog is distributed across four releases grouping nine sprints. Each release brings the product to a coherent level of maturity : R1 makes the platform usable and secure, R2 opens operational steering, R3 delivers the financial and governance value, and R4 consolidates the whole.

Release	Sprint	Scope	User stories
R1 — Foundation	Sprint 1	Technical foundation, authentication	US-01 to US-03
	Sprint 2	Dynamic RBAC, users, resources	US-04 to US-07
R2 — Operations	Sprint 3	Projects and lifecycle	US-08 to US-10
	Sprint 4	Teams and workload	US-11 to US-14
R3 — Finance	Sprint 5	Billing and missions	US-15 to US-18
	Sprint 6	Governance	US-19 to US-21
	Sprint 7	Internal quote and KPI engine	US-22 to US-25
R4 — Delivery	Sprint 8	Consolidation, tests, deployment	US-26
	Sprint 9	Containerization and continuous delivery	Technical sprint, no user story

TABLEAU 2.5 : Distribution of the backlog across releases and sprints

Conclusion

This chapter established what the platform must do, for whom, and under which rules, and translated those requirements into a product backlog distributed across the development sprints. The next chapter presents the architecture and the design models, the data model and the database schema, on which this construction rests.

SYSTEM ARCHITECTURE AND DESIGN

Plan

1	Architectural Style	23
2	Cross-Cutting Design Decisions	25
3	Domain Model	26
4	Database Design	26
5	Deployment Architecture	28
6	Technical Environment	29

Introduction

This chapter translates the requirements of Chapter 2 into a technical design. It presents the architectural style retained, the layered organisation of the application, the cross-cutting design decisions that structure every module, the domain model, and the relational schema that supports it. These choices are made once here and are then applied uniformly by the nine sprints of Chapters 4 to 7.

3.1 Architectural Style

3.1.1 A Three-Tier Modular Monolith

The platform follows a classical **three-tier** organisation (presentation, application, data), deployed as a **modular monolith** : a single deployable artefact whose code is partitioned into independent functional modules.

This choice deserves justification, since a microservice architecture is often the default expectation for a management platform. Three arguments led to the monolith :

- **Transactional consistency.** The PMS domain is strongly coupled : computing an indicator reads workload, billing, cost rates and the internal quote at once. Keeping these aggregates in a single transactional boundary avoids distributed transactions for no functional gain.
- **Deployment scale.** The platform targets the internal use of one company. The operational cost of a distributed architecture would not be repaid by the load.
- **Reversibility.** Partitioning the code into modules with explicit boundaries preserves the option of extracting a module into a separate service later, should the need arise.

Figure 3.1 shows the ten functional packages and their dependencies.

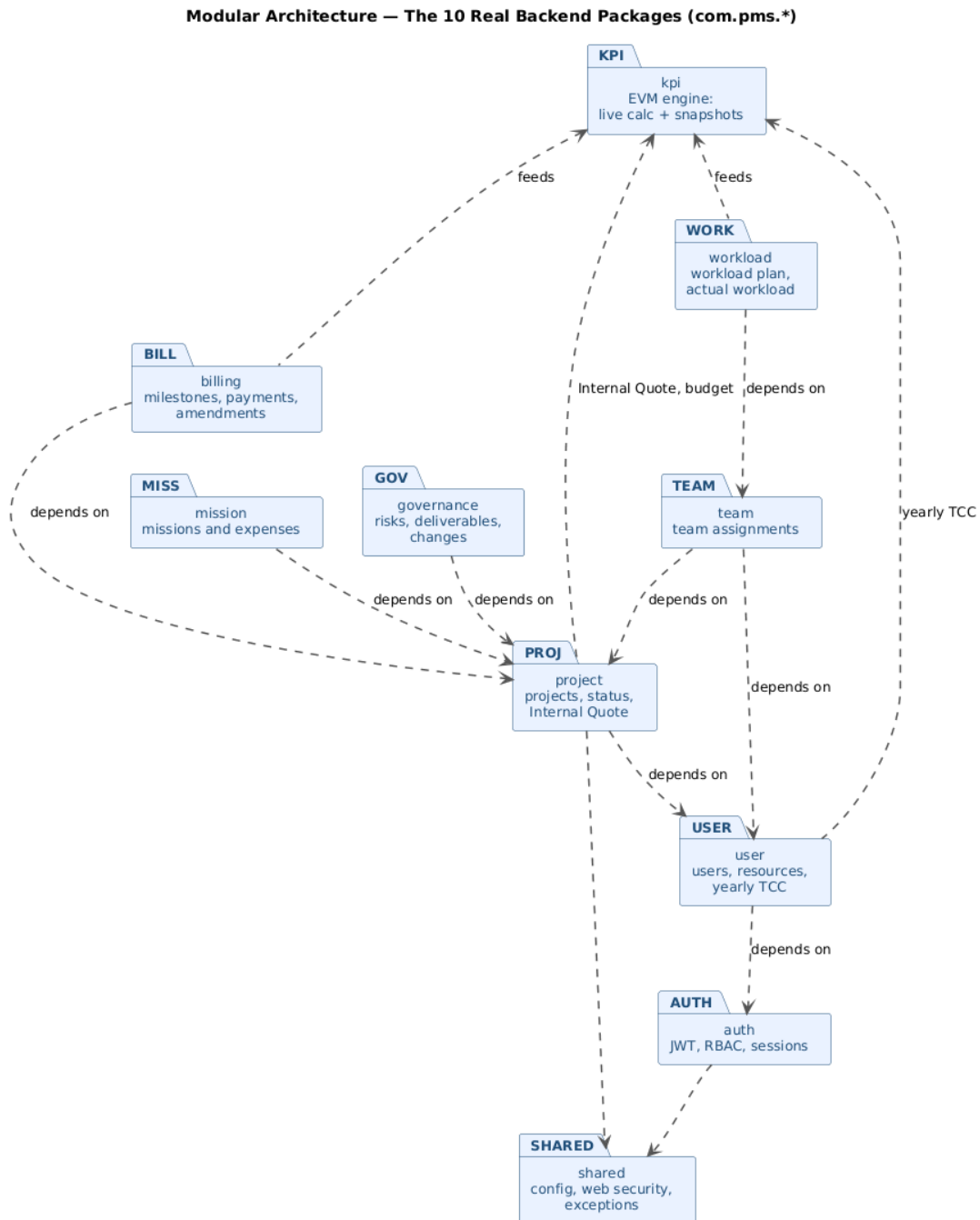


FIGURE 3.1 : Functional modules and their main dependencies

The dependency graph is deliberately acyclic and converges towards the **kpi** module, which consumes the others without being consumed by any : the indicator engine is the end of the chain, never a prerequisite.

3.1.2 Layered Organisation

Each module exposes the same internal layering :

- **Controller** — REST entry point, request validation, authorization annotation.
- **Service** — business logic, transaction boundary, rule enforcement.

- **Repository** — persistence access, implicit filtering of logically deleted rows.
- **Domain** — entities and enumerations carrying the business invariants.

Conversions between entities and data transfer objects are handled by generated mappers, so that persistence structures are never exposed directly by the API.

3.2 Cross-Cutting Design Decisions

3.2.1 Authentication and Session Revocation

Authentication rests on a short-lived JWT access token paired with an opaque refresh token stored in an `HttpOnly` cookie. Revocation does not rely on a blacklist but on a version counter carried by the token and compared on every request, which allows a session to be cut off instantly without distributed state. The mechanism is detailed in Sprint 1.

3.2.2 Dynamic Authorization

The “Role $\rightarrow$ Permission $\rightarrow$ Module” model presented in Section 2.5 is enforced by annotating every endpoint with the permission code it requires. No role name appears in application code. The role–permission matrix lives in the database, which makes the authorization policy configurable data. This is the single most structuring decision of the project and is validated in Sprint 2.

3.2.3 A Hybrid Indicator Engine

Financial indicators could be either recomputed on demand or persisted at each change. The design retained is **hybrid** :

- indicators are **recomputed on read** from live data, which guarantees that a dashboard never displays a stale figure ;
- a **monthly snapshot** freezes the official state of each review, which preserves the history required to plot the evolution of a project.

This mirrors the reference workbook, where the dashboard reads live values while the statistics sheet keeps one column per month. Amounts derived from the internal quote are always computed at read time and never stored, so that no commercially sensitive value is duplicated in the database.

3.2.4 Multi-Currency Handling

Projects may be contracted in a currency different from the reporting currency. Every amount therefore carries its currency, and conversions use a **dated rate** : each conversion applies the rate at

the business date of the amount : the invoice date for billing, the month for monthly costs, the mission date for travel. Mission cost components are converted before summation, so that currencies are never mixed within a total.

3.2.5 Auditability and Logical Deletion

Every business table carries creation and modification metadata and an `active` / `deleted_at` pair. No row is physically removed : a deletion is a state change. This satisfies the auditability requirement and preserves the history on which the indicators depend.

3.3 Domain Model

Figure 3.2 gives the overall map of the domain. The project is the aggregation root : most business entities exist only within a project and are composed by it, while users, roles, permissions and resources form an independent reference core.

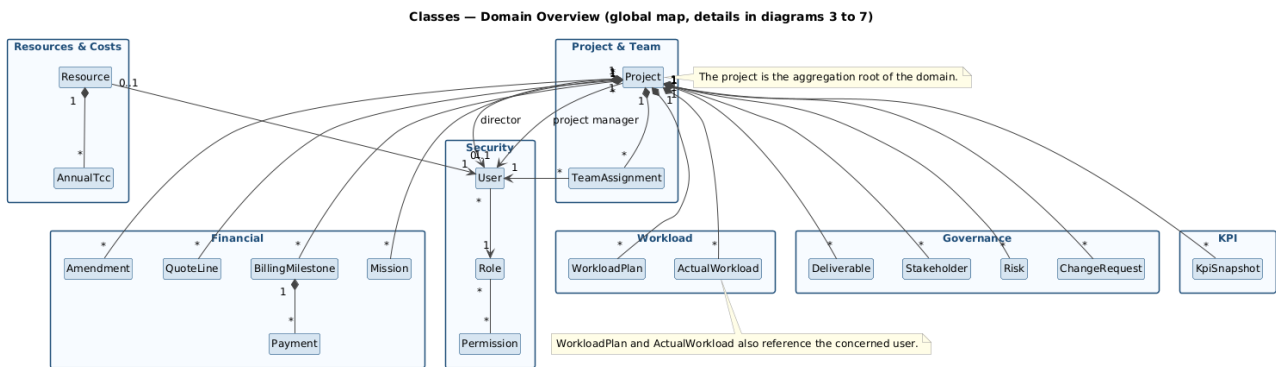


FIGURE 3.2 : Domain overview — entities grouped by sub-domain

Each sub-domain is detailed by its own class diagram, presented in the sprint that builds it : security in Sprint 2 (Figure 3.3), projects and teams in Sprint 3, workload and indicators in Sprint 7, financial management in Sprint 5 and governance in Sprint 6.

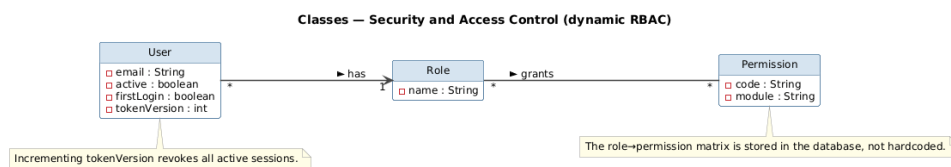


FIGURE 3.3 : Class diagram — security and access control

3.4 Database Design

3.4.1 Modelling Conventions

The schema targets PostgreSQL and follows uniform conventions :

- tables in **snake_case**, plural; identity primary key **id**;
- foreign keys named **<entity>_id**; timestamps in **timestamp_tz**;
- amounts stored as **numeric(18,3)** with an explicit currency column, never as floating point, to avoid rounding drift on financial aggregates;
- months stored as the **first day of the month**, making the workload a true time series;
- enumerations expressed as **varchar** constrained by **CHECK**;
- audit columns and logical deletion on every business table.

3.4.2 Logical Model

The core of the schema is the dynamic access control : **users** references **roles**, and **roles** relates to **permissions** through the **role_permissions** association. A project references its director and groups its child entities : project manager assignment (a single active one at a time), team assignments, workload plan, actual workload, billing milestones, missions, governance registers and KPI snapshots. A resource carries one cost rate per year and may be linked to a user account.

Domain	Main tables	Module
Security / RBAC	users , roles , permissions , role_permissions	Authentication & RBAC
Reference data	currencies , exchange_rates , parameters , risk scales	Configuration
Resources	resources , tcc	Cost rate management
Projects	projects , project_manager_assignments	Project management
Teams	team_assignments , stakeholders	Teams
Workload	workload_plan , actual_workload	Planning / Actuals
Billing	billing_milestones , payments , avenants	Billing
Missions	missions	Mission management
Governance	risks , deliverables , change_requests , actions	Governance
Indicators	kpi_snapshots	KPI & Reporting

Domain	Main tables	Module
--------	-------------	--------

TABLEAU 3.1 : Catalogue of entities by domain

3.4.3 Monthly Time Series and Indicator Snapshots

The Excel workbook is a resource month grid. Rather than reproducing twenty-five month columns, the model is **normalised** : one row per (project, resource, month) in **workload_plan** and **actual_workload**. Adding a month becomes inserting a row rather than altering the schema.

The **kpi_snapshots** table stores one snapshot per (project, month), which yields both the history needed for evolution curves and fast reads for dashboards. It deliberately separates the **labour cost** (man-days cost rate) from **other costs** (missions and expenses), the total aggregating the two. The original workbook blurred that distinction, which matters when analysing why a project drifts.

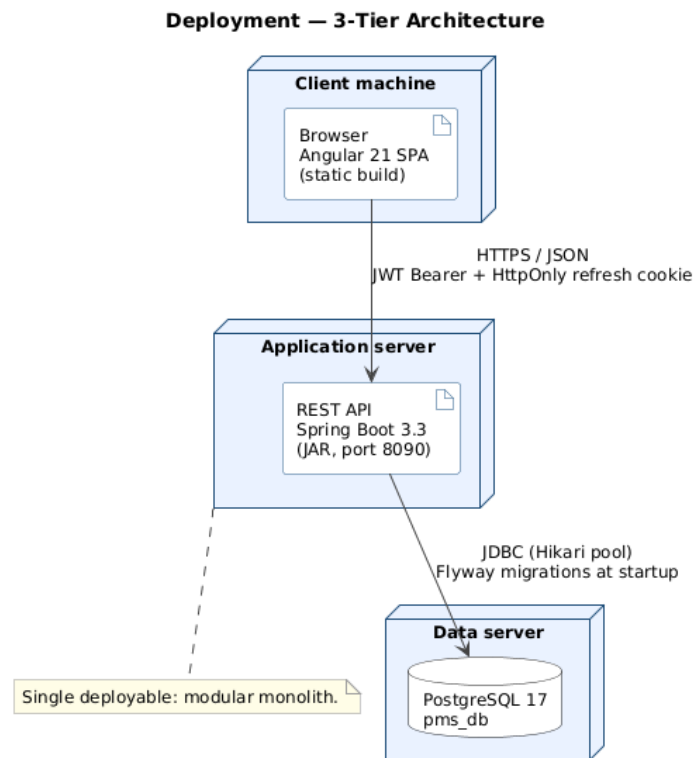
3.4.4 Reference Data and Migration Strategy

A seed dataset creates the four roles, the full permission catalogue, the default role–permission matrix, the currencies, the business parameters, the risk scales and a bootstrap administrator.

The schema is managed exclusively by **Flyway** versioned SQL migrations, with the ORM in **validate** mode : the entities must match the schema, never generate it. The migrations constitute the living documentation of the database and make every schema change reviewable and reproducible.

3.5 Deployment Architecture

Figure 3.4 shows the target deployment : a browser running the single-page application, an application server exposing the REST API, and a database server. The whole platform is a single deployable artefact, consistent with the modular monolith decision.

**FIGURE 3.4 :** Deployment architecture

3.6 Technical Environment

Layer	Technologies
Backend	Java, Spring Boot, Spring Security, JPA/Hibernate, MapStruct
Database	PostgreSQL, Flyway migrations
Frontend	Angular, TypeScript, standalone components and signals
Security	JWT access and refresh tokens, BCrypt password hashing
Modelling	PlantUML for the UML diagrams, versioned as source

TABLEAU 3.2 : Technical environment of the project

Conclusion

The architecture retained, a three-tier modular monolith with dynamic authorization, a hybrid indicator engine and a normalised time-series schema, answers the requirements of Chapter 2 while remaining proportionate to the scale of the target deployment. These decisions are now applied : the following chapters present the progressive construction of the platform, release by release and sprint by sprint.

RELEASE 1 — FOUNDATION AND SECURITY

Plan

- 1 Sprint 1 — Technical Foundation and Authentication 31
- 2 Sprint 2 — Dynamic Access Control, Users and Resources 34

Introduction

This chapter presents the first release of the PMS platform. Its purpose is to make the application usable and secure before any business functionality is built on top of it. It gathers two sprints : Sprint 1 establishes the technical foundation and authentication, Sprint 2 delivers the dynamic access control together with the management of users and resources.

At the end of this release, the platform authenticates its users, applies a configurable authorization policy, and holds the reference data (users, resources, cost rates) that the operational modules of Release 2 will consume.

4.1 Sprint 1 — Technical Foundation and Authentication

4.1.1 Sprint Objective

Establish the technical skeleton of the backend application and deliver a complete, secure authentication chain. No business functionality is addressed in this sprint : the goal is to obtain a running, testable and secured application shell.

4.1.2 Functional Scope

This sprint covers user stories US-01 to US-03 of the backlog (Table 2.4) : signing in, the forced password change on first login, and the immediate revocation of active sessions.

4.1.3 Features Implemented

- Modular Spring Boot application skeleton.
- Automatic auditing and logical deletion shared by all business entities.
- Versioned database schema management through Flyway migrations.
- Login by e-mail and password with JWT access token and refresh token.
- Immediate session revocation by token version.
- Mandatory password change on first login.

4.1.4 Technical Realisation

4.1.4.1 Modular monolith structure

The application follows a **modular monolith** architecture : the code is organised into independent functional packages, each exposing its own Controller → Service → Repository → Domain layering. The

modules are `auth`, `user`, `project`, `team`, `workload`, `kpi`, `billing`, `mission`, `governance` and `shared`. This structure allows each subsequent sprint to add its module without touching the previous ones.

4.1.4.2 Base entity and automatic auditing

All business entities inherit from `BaseEntity`, which centralises the auto-generated primary key, the `createdAt` / `updatedAt` timestamps managed by `@PrePersist` and `@PreUpdate`, and the `deleted` marker implementing logical deletion. No data is ever physically erased, and every repository implicitly filters on `deleted = false`. This guarantees the traceability required by the auditability requirement (NFR-008).

4.1.4.3 Schema management through Flyway

The database structure is managed exclusively by Flyway. Hibernate never emits implicit DDL : the property `spring.jpa.hibernate.ddl-auto` is set to `validate`. The versioned migrations are idempotent and constitute the living documentation of the schema.

4.1.4.4 Token issuance

On `POST /api/auth/login`, the authentication service verifies the e-mail and compares the supplied password with the BCrypt hash (strength 12), checks that the account is active, then issues :

- a short-lived **JWT access token** signed with `HMAC-SHA512`, carrying the e-mail, the token version and the list of permission codes ;
- an opaque **refresh token** placed in an `HttpOnly` / `Secure` / `SameSite=Strict` cookie.

Separating the two tokens, access token in memory and refresh token in a cookie, protects against XSS attacks : the more powerful refresh token is never reachable from JavaScript.

4.1.4.5 Immediate revocation by token version

Revocation rests on a `token_version` field persisted in the `users` table. The current version is embedded in every issued token. On each request, the `JwtAuthenticationFilter` compares the version carried by the token with the value read from the Caffeine cache ; if they differ, the request is rejected with `401 Unauthorized`.

Incrementing `token_version` therefore invalidates every active session of that identity instantly : on logout, and on password change, which closes sessions opened on other devices. The cache avoids a database access on every request while keeping consistency without distributed state.

4.1.4.6 Forced password change on first login

When an account is created, the administrator sets a temporary password and `first_login` is set to `true`. The token issued at the first connection carries a restricted right : only `POST /api/auth/change-password` is authorised. Once the password is changed, `first_login` becomes `false` and an unrestricted token is issued.

4.1.5 Tests and Validation

The authentication chain is covered by automated backend tests verifying the nominal path and the rejection paths.

Test case	Steps	Expected result
Successful login	Submit a valid e-mail and password on an active account	HTTP 200, access token returned and refresh cookie set
Wrong password	Submit a valid e-mail with an incorrect password	HTTP 401, no token issued
Inactive account	Submit valid credentials on a deactivated account	HTTP 401, access refused
First login	Call a business endpoint with a first-login token	Access refused ; only the password change endpoint is reachable
Revocation	Call an endpoint with a token issued before a password change	HTTP 401, stale token version detected

TABLEAU 4.1 : Test cases — Sprint 1

4.1.6 Sprint Outcome

At the end of Sprint 1 the platform is a running, secured application shell : a user can sign in, is forced to choose a personal password, and their sessions can be cut off immediately. The auditing, logical deletion and schema-migration mechanisms are in place and will be inherited by every module built afterwards.

4.1.7 Transition to the Next Sprint

The application authenticates its users but does not yet distinguish what each of them may do : the token carries permission codes that nothing populates yet. Sprint 2 therefore builds the dynamic

authorization model that fills this gap, together with the management of the users and resources it applies to.

4.2 Sprint 2 — Dynamic Access Control, Users and Resources

4.2.1 Sprint Objective

Deliver the authorization model that structures the whole platform, the “Role $\rightarrow$ Permission $\rightarrow$ Module” chain introduced in Section 2.5, and the administration screens that feed it with reference data.

4.2.2 Functional Scope

This sprint covers user stories US-04 to US-07 : assigning permissions to roles, adapting the interface to the rights held, managing user accounts, and recording resources with their yearly cost rate.

4.2.3 Features Implemented

- Permission–Role–User model persisted in the database.
- Propagation of permissions to Spring Security and protection of every endpoint.
- Navigation menu built dynamically from the rights held.
- User account management : creation, modification, activation and deactivation.
- Resource management and entry of the yearly loaded cost rate (TCC).

4.2.4 Technical Realisation

4.2.4.1 Permission–Role–User model

Access control is **entirely dynamic** : permissions are not hard-coded in the application. The `role_permissions` table associates a set of permissions with a role. A user holds a single active role, but that role can be reconfigured at any time by the administrator without redeployment.

A dedicated migration initialises the four default roles (Administrator, Director, Project Manager, Developer) with eighteen permission codes spread over seven modules (`ADMIN`, `PROJET`, `EQUIPE`, `CHARGE`, `FACTURATION`, `KPI`, `GOV`).

4.2.4.2 Propagation to Spring Security

The `JwtAuthenticationFilter` reads the permission codes from the token and converts them into `SimpleGrantedAuthority` objects. Every REST entry point is then protected by an annotation of the form :

```
@PreAuthorize("hasAuthority('CREATE_PROJECT')")
```

The absolute rule is that **no role name is ever tested in application code** : only permission codes are used. This is what makes the authorization policy a matter of data rather than of source code.

4.2.4.3 Dynamic menu

The `GET /api/me/context` endpoint returns, for the authenticated user, the list of their permissions together with their profile information. The Angular frontend uses this response to build the navigation menu : an item is displayed only if the user holds the required permission code. The interface therefore reflects the authorization policy automatically, with no duplicated rule on the client side.

4.2.4.4 Users and resources

The **user** module manages accounts (creation by the administrator with a temporary password, modification, activation and deactivation, role assignment) and the human resources with their daily rate and loaded cost rate. The distinction between the two notions is deliberate : a resource represents a cost-bearing capacity, which may or may not be linked to a platform account. Object conversions between entities and DTOs are handled by `MapStruct` mappers.

The loaded cost rate is recorded **per resource and per year**, so that the cost of a man-day uses the rate of the year in which the effort was booked, a requirement drawn directly from the Excel workbook.

4.2.5 Tests and Validation

Validating this sprint means proving that authorization really is data-driven : that granting a permission changes behaviour without any code change.

Test case	Steps	Expected result
Permission held	Call a protected endpoint with a role holding the required permission	HTTP 200, operation performed

Test case	Steps	Expected result
Permission missing	Call the same endpoint with a role that does not hold it	HTTP 403, operation refused
Permission granted at runtime	Add the permission to the role, then repeat the call	HTTP 200 without redeployment
Menu adaptation	Sign in with two different roles	Each interface exposes only the authorised sections
Yearly cost rate	Book effort in two different years	Each computation uses the rate of its own year

TABLEAU 4.2 : Test cases — Sprint 2

4.2.6 Sprint Outcome

The platform now knows not only *who* the user is but *what they may do*, and the authorization policy can be modified by an administrator without touching the code. The reference data required by the business modules (users, resources, yearly cost rates) is in place.

4.2.7 Transition to the Next Sprint

Release 1 delivers a secured and administrable platform, but it manages no project yet. Release 2 opens operational steering, starting in Sprint 3 with the central entity of the whole domain : the project.

Conclusion

Release 1 established the foundations on which the entire platform rests : a modular application skeleton with auditing and controlled migrations, a robust authentication chain, and above all a dynamic authorization model that makes the security policy configurable data rather than compiled code. The next release exploits this foundation to deliver the operational management of projects and workload.

RELEASE 2 — PROJECT AND OPERATIONAL MANAGEMENT

Plan

1	Sprint 3 — Project Management and Lifecycle	38
2	Sprint 4 — Teams and Workload Management	41

Introduction

Release 1 delivered a secured and administrable platform, but one that managed no project. This release opens operational steering. Sprint 3 introduces the central entity of the whole domain — the project, with its controlled lifecycle and its budget ; Sprint 4 populates it with a team and with the workload data on which every later indicator depends.

At the end of this release, a project can be created, staffed, planned, and its actual consumption recorded : everything the financial engine of Release 3 will need as input.

5.1 Sprint 3 — Project Management and Lifecycle

5.1.1 Sprint Objective

Deliver the management of projects : creation with their contractual and financial identity, assignment of a project manager, and a lifecycle that the application enforces rather than merely documents.

5.1.2 Functional Scope

This sprint covers user stories US-08 to US-10 : creating and configuring a project, controlling its status transitions, and assigning its project manager.

5.1.3 Features Implemented

- Project creation with code, client, funder, engagement type, currency and budget.
- Controlled status lifecycle with server-side enforcement.
- Effective budget derived from the initial budget and its amendments.
- Project manager assignment, with a single active assignment at a time.
- Project archiving, distinct from closure.

5.1.4 Technical Realisation

5.1.4.1 Project identity

The **project** module reproduces the identification sheet of the reference workbook : project code, client, funder, business model, engagement type (fixed price or time & material), project currency with its contractual exchange rate, initial budget, sold workload and provisions. The class model of the module is given in Figure 5.1.

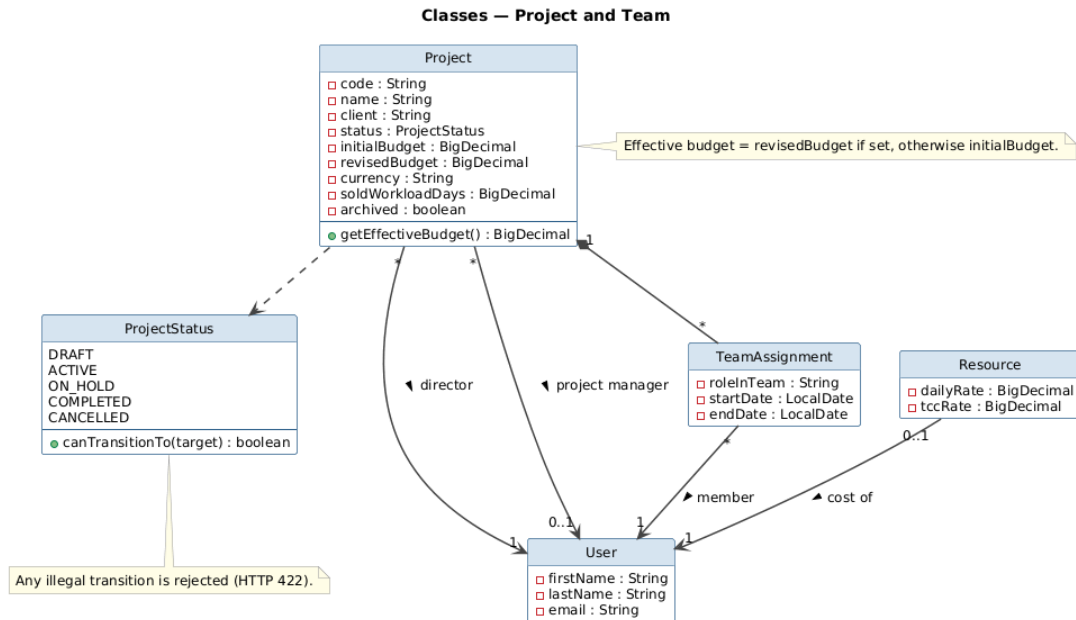


FIGURE 5.1 : Class diagram — projects and teams

5.1.4.2 A lifecycle enforced by the domain

A project moves through five states : DRAFT, ACTIVE, ON_HOLD, COMPLETED and CANCELLED. Not every transition is legal : a completed project cannot return to draft, a cancelled project is terminal.

The rule is carried by the enumeration itself, through a `canTransitionTo` operation that each status answers for its own legal successors. The service layer consults it before any status change and rejects an illegal transition with HTTP 422 Unprocessable Entity. Placing the rule in the domain rather than in the controller guarantees that no future entry point can bypass it.

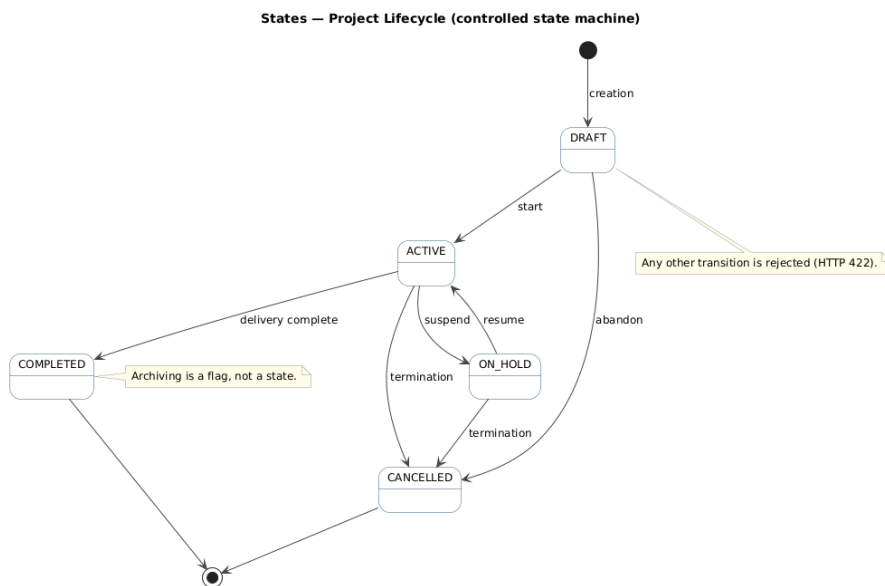


FIGURE 5.2 : State diagram — project lifecycle

Archiving is deliberately **not** a state but a flag on a completed project : an archived project

leaves the active lists while remaining restorable, which a terminal state would not allow.

5.1.4.3 Effective budget

A project carries an initial budget and, once amendments exist, a revised budget. Every computation uses the **effective budget** : the revised budget when it is defined, the initial budget otherwise. This value has a **single writer**, the amendment, which prevents the two figures from drifting apart, a failure mode observed in the original workbook where the budget was edited in several places.

5.1.4.4 Project manager assignment

A project has exactly one active project manager at any time, but keeps the history of previous assignments. Assigning a new manager therefore closes the current assignment rather than overwriting it, preserving the traceability required for auditing.

5.1.5 Tests and Validation

Validation focuses on the lifecycle, since it is the rule most likely to be violated by a client application.

Test case	Steps	Expected result
Legal transition	Move a project from DRAFT to ACTIVE	HTTP 200, status updated
Illegal transition	Move a project from COMPLETED back to DRAFT	HTTP 422, status unchanged
Terminal state	Attempt any transition out of CANCELLED	HTTP 422
Effective budget	Add an amendment then read the project	Computations use the revised budget
Manager assignment	Assign a second project manager	Previous assignment closed, one active manager remains

TABLEAU 5.1 : Test cases — Sprint 3

5.1.6 Sprint Outcome

Projects exist, carry their contractual and financial identity, and follow a lifecycle the application enforces. The invalid states that a spreadsheet cannot prevent are now structurally impossible.

5.1.7 Transition to the Next Sprint

A project without a team consumes nothing and produces no indicator. Sprint 4 therefore staffs the project and captures the effort planned and consumed, the raw material of the whole financial model.

5.2 Sprint 4 — Teams and Workload Management

5.2.1 Sprint Objective

Allow a project manager to build the project team, plan the monthly workload, and let developers record the effort actually consumed, with the guarantees of integrity the indicators will depend on.

5.2.2 Functional Scope

This sprint covers user stories US-11 to US-14 : building the team, planning the workload, submitting actual workload, and validating it.

5.2.3 Features Implemented

- Assignment and removal of team members, with preserved history.
- Monthly workload planning per resource.
- Submission of actual workload, restricted to assigned projects.
- Validation of submitted workload by the project manager.
- Immutability of a validated period.

5.2.4 Technical Realisation

5.2.4.1 Team assignment

A team assignment links a user to a project with a role in the team and a validity period. A developer may belong to several projects simultaneously. Removing a member closes the assignment rather than deleting it, so that past workload remains attributable.

Assignment is protected by **two distinct checks**, illustrated in Figure 5.3 : the caller must hold the `ASSIGN_DEVELOPER` permission *and* the target project must fall within their scope. The permission answers “what may I do” ; the scope answers “on which projects”. Conflating the two would let a project manager act on a project that is not theirs.

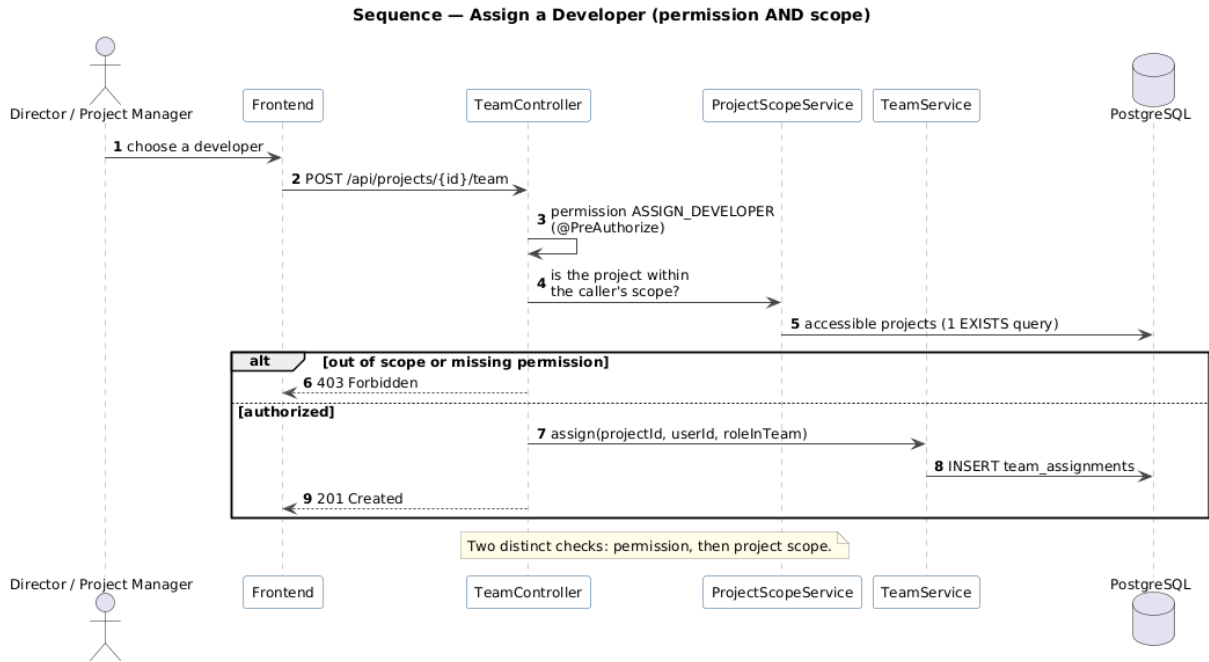


FIGURE 5.3 : Sequence — assigning a developer, permission and scope

5.2.4.2 Two kinds of workload

The model distinguishes the **workload plan** (what is forecast) from the **actual workload** (what was consumed). Both are stored as a monthly time series, one row per project, resource and month, rather than as a wide grid of month columns.

5.2.4.3 Period normalisation and uniqueness

A month is stored as the first day of that month. This normalisation is what makes uniqueness enforceable : without it, two entries for the same month recorded on different days would coexist silently. A unique partial index guarantees a single retained value per cell, and it is that value which feeds the indicator engine.

5.2.4.4 Validation and immutability

A developer submits their own workload, on assigned projects only. The project manager then validates it. Once a period is validated it becomes **immutable** : a later correction cannot silently rewrite a month on which a monthly review has already been based. This constraint is the direct answer to the traceability weakness identified in the Excel process.

5.2.5 Tests and Validation

Test case	Steps	Expected result
Authorised assignment	Assign a developer with the permission, on a project in scope	HTTP 201, assignment recorded
Out-of-scope assignment	Assign on a project outside the caller's scope	HTTP 403
Submission on an unassigned project	Submit workload on a project the developer is not assigned to	Refused
Duplicate month	Submit twice for the same project, resource and month	A single retained value
Validated period	Modify a period already validated	Refused, period immutable

TABLEAU 5.2 : Test cases — Sprint 4

5.2.6 Sprint Outcome

Projects now carry a team, a forecast and a measured consumption, all captured under integrity constraints that a spreadsheet cannot express : no duplicate month, no entry on an unassigned project, no silent rewriting of a validated period.

5.2.7 Transition to the Next Sprint

The cost side of the project is now measurable. The revenue side is not : nothing yet records what is invoiced, nor the expenses that weigh on the margin. Release 3 delivers the financial and governance value of the platform, starting with billing and missions.

Conclusion

Release 2 turned a secured shell into an operational tool : projects with an enforced lifecycle, teams with traceable assignments, and workload captured as a reliable monthly time series. The data required by the financial engine is now available and trustworthy, which is the precondition for the indicators built in the next release.

RELEASE 3 — FINANCIAL MANAGEMENT AND GOVERNANCE

Plan

1	Sprint 5 — Billing and Missions	45
2	Sprint 6 — Project Governance	47
3	Sprint 7 — Internal Quote and Indicator Engine	49

Introduction

This release delivers the value for which the platform was commissioned : replacing the financial logic of the Excel workbook. Sprint 5 records the revenue side (billing milestones, payments, amendments) and the mission expenses that weigh on the margin. Sprint 6 adds the governance registers that document the life of a project. Sprint 7 closes the loop with the internal quote and the indicator engine, which consumes everything built since Release 1.

6.1 Sprint 5 — Billing and Missions

6.1.1 Sprint Objective

Record what the project earns and what it spends outside labour : billing milestones with their payments, contractual amendments, and business-trip costs.

6.1.2 Functional Scope

This sprint covers user stories US-15 to US-18 : defining billing milestones, recording payments, recording amendments, and recording missions with their expenses.

6.1.3 Features Implemented

- Billing milestones expressed as a percentage of the contract, with a lifecycle.
- Payments recorded against a milestone.
- Contractual amendments adjusting the revised budget.
- Missions with multi-currency cost components and automatic total.

6.1.4 Technical Realisation

6.1.4.1 Billing milestones

A milestone belongs to one project and carries a label, a percentage of the contract, an amount, a due date and a status. The amount is derived from the effective budget and the percentage, so that an amendment automatically propagates to the milestones still to be invoiced.

The structural rule inherited from the workbook is enforced in the service layer : the **sum of milestone percentages may not exceed 100 %** of the contract. Attempting to exceed it is rejected rather than silently accepted, which the spreadsheet allowed.

6.1.4.2 Milestone lifecycle

A milestone moves through three states : **PREVU** (planned), **FACTURE** (invoiced) and **PAYE** (paid). Invoicing records the invoice date ; a payment recorded against an invoiced milestone moves it to paid. Payments are kept as individual rows rather than as a single settled amount, so that partial settlements remain visible.

6.1.4.3 Amendments

An amendment carries a reference, an amount and an impact on the sold workload. It is the **single writer** of the revised budget introduced in Sprint 3 : no other operation may modify it. This constraint is what keeps the contractual and financial history coherent.

6.1.4.4 Missions and multi-currency

A mission belongs to a project and an employee, and its total cost is the sum of its components — per diem, ticket, accommodation, transport, stamp. Because a mission may be incurred in a currency different from the reporting currency, each component is converted **before** summation using the rate at the mission date. Summing first and converting afterwards would mix currencies and distort the total.

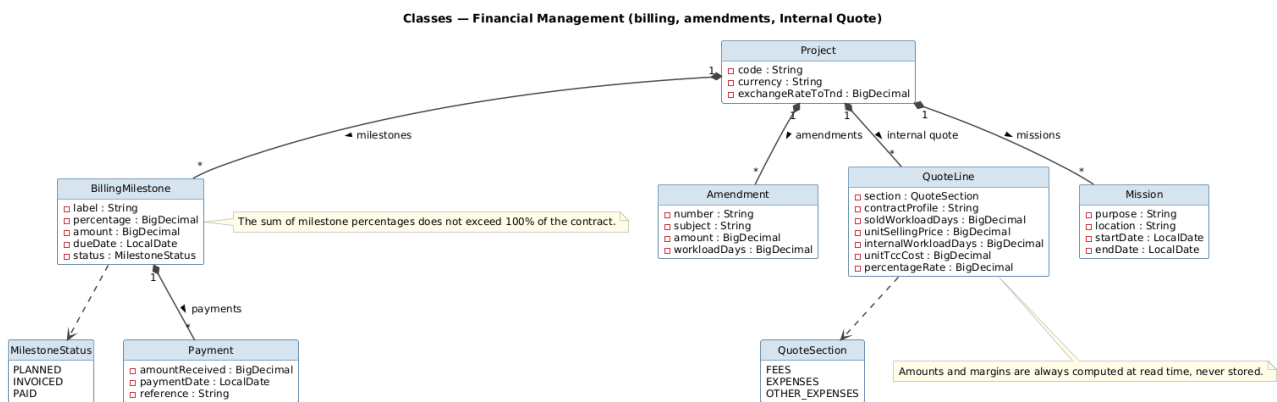


FIGURE 6.1 : Class diagram — financial management

6.1.5 Tests and Validation

Test case	Steps	Expected result
Percentage ceiling	Create milestones whose percentages exceed 100 %	Rejected, ceiling enforced
Milestone lifecycle	Invoice then settle a milestone	Status moves PREVU → FACTURE → PAYE

Test case	Steps	Expected result
Partial payment	Record two payments on one milestone	Both retained, settlement traceable
Amendment impact	Add an amendment	Revised budget updated, milestones recomputed
Mission in a foreign currency	Record components in another currency	Each converted at the mission date before summation

TABLEAU 6.1 : Test cases — Sprint 5

6.1.6 Sprint Outcome

The revenue and expense sides of a project are now recorded under the same rules the workbook applied, but enforced rather than merely intended. Together with the labour cost from Sprint 4, the platform holds every input the margin computation requires.

6.1.7 Transition to the Next Sprint

Financial tracking answers “how much” ; it does not answer “what is threatening this project”. Sprint 6 adds the governance registers that document risks, deliverables and changes.

6.2 Sprint 6 — Project Governance

6.2.1 Sprint Objective

Deliver the governance registers of the reference workbook : risks, deliverables, stakeholders and change requests, so that the qualitative life of a project is documented alongside its financial figures.

6.2.2 Functional Scope

This sprint covers user stories US-19 to US-21.

6.2.3 Features Implemented

- Risk register with probability, impact, status and mitigation plan.
- Deliverable register with due date and delivery status.
- Stakeholder register with role and influence.
- Change request register with priority and decision status.

6.2.4 Technical Realisation

The four registers share the same shape : each is composed by the project, is governed by a small business enumeration, and is edited by the project manager. Their value lies less in their individual complexity than in being attached to the project rather than kept in a separate document, which is what the spreadsheet process could not guarantee.

The **deliverable register** carries a functional responsibility beyond documentation : the delivery rate used by the indicator engine of Sprint 7 is derived from it, as the ratio of delivered and validated deliverables to planned deliverables. This is why deliverables carry a controlled status rather than a free-text field.

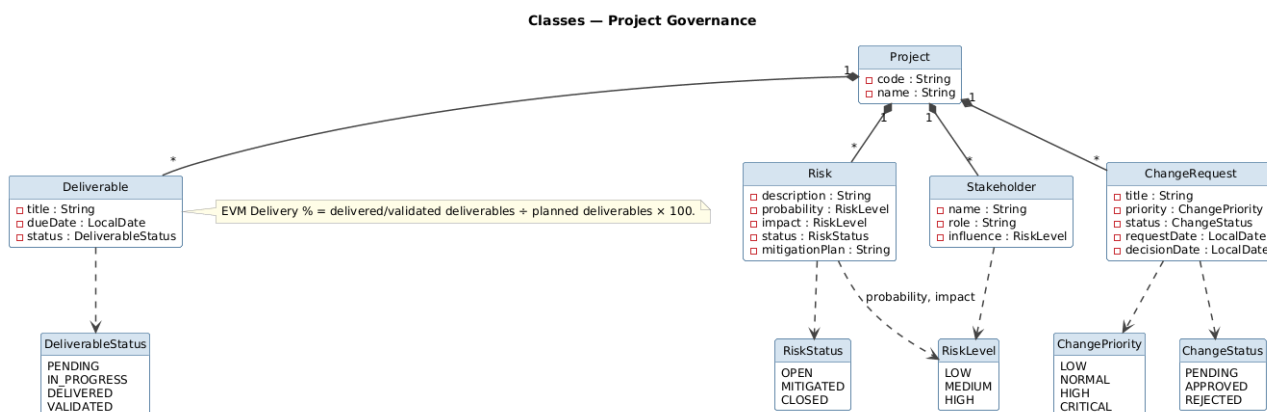


FIGURE 6.2 : Class diagram — project governance

6.2.5 Tests and Validation

Test case	Steps	Expected result
Register creation	Add a risk, a deliverable, a stakeholder and a change request	All attached to the project
Deliverable status	Move a deliverable to delivered	Delivery rate recomputed accordingly
Authorization	Access a register without the governance permission	HTTP 403
Logical deletion	Delete a register entry	Entry deactivated, history preserved

TABLEAU 6.2 : Test cases — Sprint 6

6.2.6 Sprint Outcome

The qualitative dimension of project steering is now captured in the platform, and the deliverable register provides the delivery rate the indicator engine needs.

6.2.7 Transition to the Next Sprint

Every input is now available : labour cost, other costs, invoiced revenue, delivery progress. Sprint 7 assembles them into the indicators that were the original purpose of the project.

6.3 Sprint 7 — Internal Quote and Indicator Engine

6.3.1 Sprint Objective

Deliver the economic baseline of a project, the internal quote, and the engine that computes the earned-value indicators, together with the dashboards that expose them.

6.3.2 Functional Scope

This sprint covers user stories US-22 to US-25 : defining the internal quote, computing the indicators automatically, freezing a monthly review, and providing the portfolio dashboard.

6.3.3 Features Implemented

- Internal quote structured by section, with restricted access.
- Automatic computation of the earned-value indicators.
- Frozen monthly review preserving the official state of the project.
- Per-project and portfolio dashboards.

6.3.4 Technical Realisation

6.3.4.1 The internal quote

The internal quote is the economic baseline of the project : line items grouped by section (fees, expenses, other expenses), each carrying a sold quantity and unit price on the revenue side, and an internal quantity and unit cost on the cost side.

Two design decisions govern this module. First, access is restricted by a dedicated permission, because unit costs and margins are commercially sensitive. Second, **amounts and margins are never stored** : they are recomputed at read time from quantities, unit values and the exchange rate.

Persisting them would duplicate sensitive figures and risk divergence between the stored value and its inputs.

6.3.4.2 The hybrid indicator engine

The engine implements the architecture decided in Chapter 3. On each read it recomputes, from live data, the indicators of the reference workbook :

- **Actual cost** — consumed man-days valued at the cost rate of their own year ;
- **ETC** — remaining man-days valued the same way ;
- **EAC** — actual cost plus ETC ;
- **Production revenue** — contract total weighted by the earned value percentage ;
- **Accrued income** — production revenue minus the invoiced total ;
- **Margin** — effective budget minus EAC, in value and in percentage ;
- **Drift** — sold workload minus consumed minus remaining.

Nothing is written during this computation : a dashboard read has no side effect.

6.3.4.3 The frozen monthly review

The earned value percentage is a judgement, not a measurement : it is entered by the project manager during the monthly review. Submitting it creates a **snapshot** that freezes the indicators of that month, together with the highlights recorded by the manager.

This is the digital equivalent of the monthly column of the statistics sheet : live figures for steering, frozen figures for history. A snapshot is immutable, so the history of past reviews cannot be rewritten by a later correction.

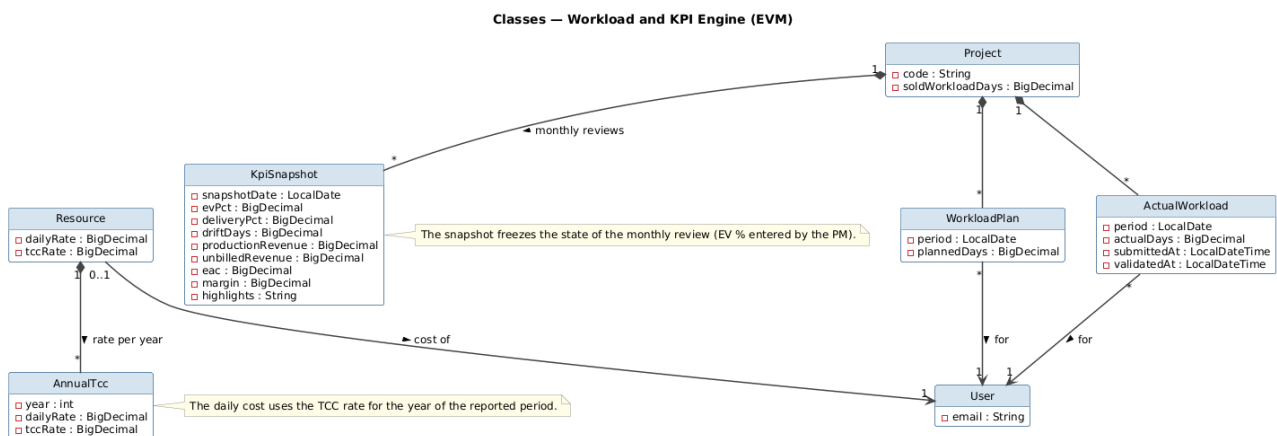


FIGURE 6.3 : Class diagram — workload and indicator engine

Figure 6.4 shows the complete chain, from workload submission to the frozen review.

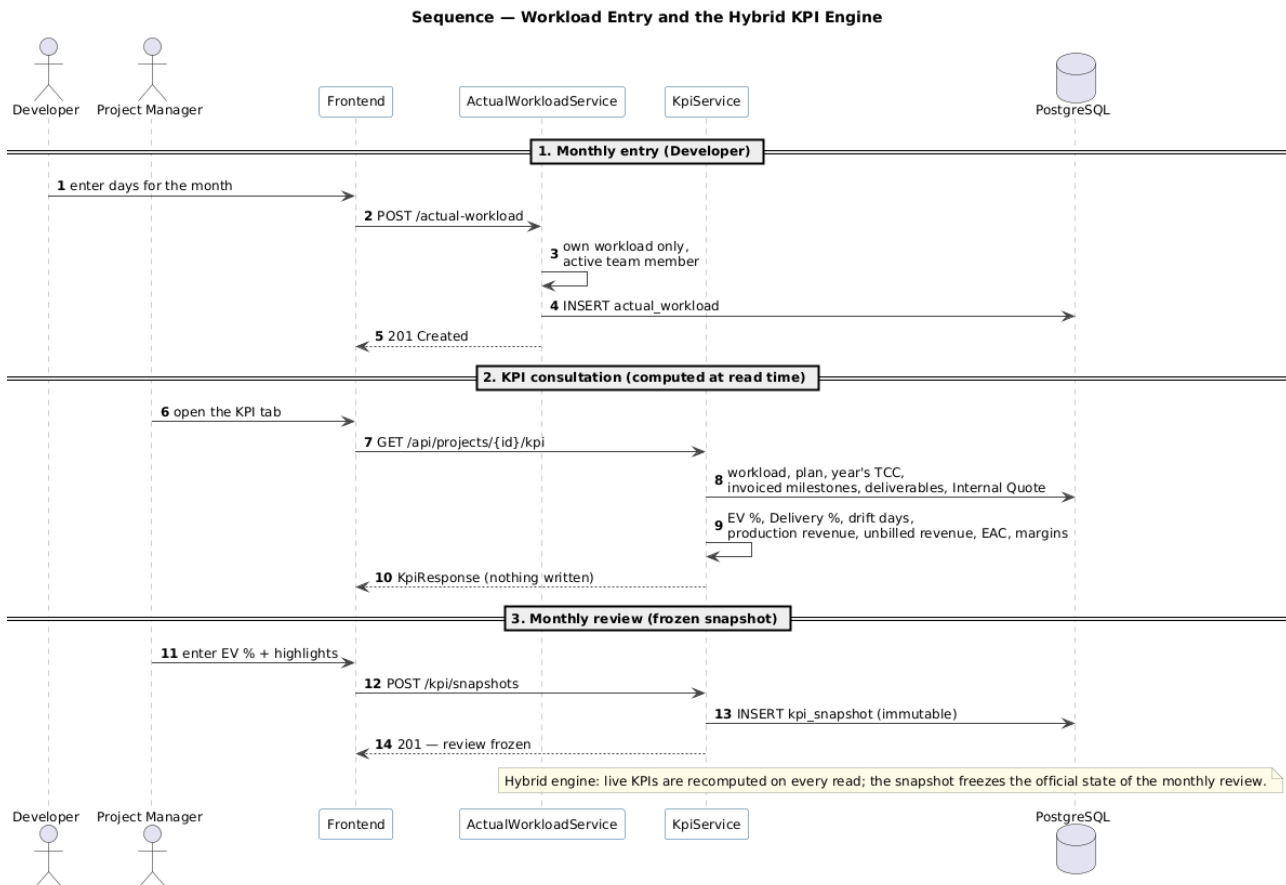


FIGURE 6.4 : Sequence — workload entry and indicator computation

6.3.5 Tests and Validation

Validation of this sprint is comparative : the indicators computed by the platform are checked against the figures produced by the reference workbook for the same inputs.

Test case	Steps	Expected result
Cost rate by year	Book effort across two years	Each year valued at its own rate
EAC computation	Compare with the workbook on identical inputs	Identical result
Read has no side effect	Read the indicators twice	No data written, identical result
Snapshot immutability	Attempt to modify a past snapshot	Refused
Financial confidentiality	Read the indicators as a developer	Financial data not exposed

Test case	Steps	Expected result
Internal quote access	Read the quote without the dedicated permission	HTTP 403

TABLEAU 6.3 : Test cases — Sprint 7

6.3.6 Sprint Outcome

The platform now reproduces the financial logic of the Excel workbook, automatically and under access control. The functional objective of the project is met : a project manager obtains their indicators without opening a spreadsheet, and a director sees the whole portfolio.

6.3.7 Transition to the Next Sprint

The functional scope is complete. What remains is to make the platform pleasant, verified and deployable, the object of Release 4.

Conclusion

Release 3 delivered the core value of PMS : billing and expenses, the governance registers, the internal quote and above all the indicator engine that replaces the two thousand formulas of the reference workbook. The hybrid design, live computation for steering and frozen snapshots for history, reproduces the strengths of the spreadsheet while removing its fragility.

RELEASE 4 — CONSOLIDATION, DELIVERY AND DEPLOYMENT

Plan

- 1 Sprint 8 — Interface Consolidation, Testing and Deployment 54
- 2 Sprint 9 — Containerization and Continuous Delivery 57

Introduction

The functional scope of the platform was completed at the end of Release 3. This final release makes it deliverable : a coherent and accessible user interface, a verified authorization policy, a security review, and a reproducible deployment. It gathers two sprints : Sprint 8 consolidates the interface and verifies the platform, Sprint 9 makes it reproducibly deployable and continuously verified. Neither delivers new business functionality ; both deliver deliverability.

7.1 Sprint 8 — Interface Consolidation, Testing and Deployment

7.1.1 Sprint Objective

Bring the platform to a deliverable state : unify the user interface, verify that the security model behaves as designed, review the application against the common web vulnerabilities, and package it for deployment.

7.1.2 Functional Scope

This sprint covers user story US-26, a consistent and responsive interface, and the cross-cutting quality work that no single functional story carries.

7.1.3 Features Implemented

- Frontend application structure and session handling.
- Authorization model applied to the interface.
- Design system unifying the screens of the eight modules.
- Automated test suite covering the business rules and the access control.
- Security review against the common web vulnerabilities.
- Deployment packaging.

7.1.4 Technical Realisation

7.1.4.1 Frontend structure and session handling

The client is a single-page application organised by feature, mirroring the backend modules. The access token obtained at sign-in is kept in memory and attached to every request by an HTTP interceptor ; the refresh token remains in its `HttpOnly` cookie, unreachable from JavaScript. When a

request returns 401, the interceptor attempts a single refresh before returning the user to the sign-in screen.

7.1.4.2 Authorization applied to the interface

The interface consumes the permission list returned by `GET /api/me/context` and derives from it both the navigation menu and the visibility of individual actions. A button that the user is not entitled to use is not merely disabled : it is absent.

This must be understood correctly, and the report states it explicitly : hiding a control is an **ergonomic** measure, not a security measure. The authorization decision is taken by the server on every request, as described in Sprint 2. The client-side rule exists so that the user is never offered an action that would be refused ; it is never the thing that refuses it.

7.1.4.3 Design system and interface consolidation

The eight modules were built successively, each with its own screens. This sprint unified them behind a single design system : a token-based visual language (colours, typography, spacing, elevation) declared once and consumed by every screen, together with a set of shared components (pagination, selection, confirmation dialog, notifications) reused across the modules rather than reimplemented in each.

The consolidation also addressed accessibility and consistency : sortable table headers announcing their state, labelled icon-only controls, explicit empty states in place of blank tables, and a responsive layout. The interface being the only part of the platform its users ever see, this convergence matters as much to adoption as the correctness of the engine behind it.

7.1.4.4 Test strategy

Testing concentrates on what a regression would make expensive : the business rules and the authorization model. The automated backend suite covers the lifecycle transitions, the billing ceiling, workload uniqueness and immutability, the multi-currency conversions, and the indicator formulas compared against the reference workbook.

The access control receives dedicated attention : for each protected endpoint, the suite verifies that a caller holding the permission succeeds and that a caller without it is refused. This is what makes the “configurable authorization” claim verifiable rather than merely stated.

7.1.4.5 Security review

The application was reviewed against the common categories of web vulnerability :

Category	Measure taken
Broken access control	Every endpoint annotated with its required permission ; scope checked independently of permission ; no role name tested in code
Cryptographic failures	Passwords hashed with BCrypt ; tokens signed ; refresh token confined to an <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> cookie
Injection	Parameterised queries through the ORM ; no string-built SQL
Identification and session failures	Short-lived access token ; instant revocation by token version ; forced password change on first login
Sensitive data exposure	Financial data filtered by permission ; internal quote behind a dedicated permission ; computed amounts never persisted
Security misconfiguration	Schema managed by versioned migrations with the ORM in validation mode ; secrets externalised as environment variables

TABLEAU 7.1 : Security review by vulnerability category

7.1.4.6 Packaging

At the close of this sprint the platform is packaged as an executable archive for the backend and a static build for the frontend, with the database schema brought up to date by the migration tool at startup and all configuration supplied by environment variables.

Startup is idempotent : deploying the same artefact against an up-to-date database changes nothing, and against an older one applies exactly the missing migrations.

This packaging is sufficient to run the application, but not to *deploy* it reproducibly : it still assumes a host on which the right Java version, the right Node version and a correctly provisioned PostgreSQL instance are already present. Sprint 9 removes that assumption.

7.1.5 Tests and Validation

Test case	Steps	Expected result
Business rule suite	Run the automated backend tests	All pass

Test case	Steps	Expected result
Permission granted	Call each protected endpoint with the permission	Operation performed
Permission missing	Call each protected endpoint without it	HTTP 403
Financial confidentiality	Browse the interface as a developer	No financial figure exposed
Menu consistency	Sign in with each of the four roles	Each menu matches the permissions held
Deployment	Start the artefact against an empty and an existing database	Schema created, then migrated without loss

TABLEAU 7.2 : Test cases — Sprint 8

7.1.6 Sprint Outcome

The platform is functionally complete, verified on its business rules and its authorization model, reviewed for the common vulnerabilities, and packaged as a runnable artefact.

7.1.7 Transition to the Next Sprint

One gap remains between “it runs on my machine” and “it can be deployed” : the artefact still depends on a correctly prepared host. Sprint 9 closes that gap with containerization and a continuous integration pipeline.

7.2 Sprint 9 — Containerization and Continuous Delivery

7.2.1 Sprint Objective

Make the platform reproducibly deployable and continuously verified : package each component as a container image, orchestrate the whole stack with a single command, and automate build, test and integration checks on every change.

7.2.2 Functional Scope

This sprint delivers no new business functionality. Its scope is the delivery chain itself, and it addresses an item already open in the project’s engineering backlog : the absence of deployment documentation and of any reproducible deployment procedure.

7.2.3 Features Implemented

- Container image for the backend, built in two stages and running as a non-root user.
- Container image for the frontend, serving the production build and reverse-proxying the API.
- Orchestration of the three services with health checks and startup ordering.
- Continuous integration pipeline : build, tests, image construction, end-to-end smoke test.
- Deployment documentation and an architecture decision record superseding the previous one.

7.2.4 Technical Realisation

7.2.4.1 A reconsidered architecture decision

The project had initially recorded a decision to run natively, without containers, justified by iteration speed on an already-provisioned machine. That decision served development well but left two gaps : no reproducible deployment, and a production profile that had never actually been executed.

The decision was therefore superseded rather than silently contradicted : a new record documents the move to containers, states what it changes, and confirms what it does *not* change. In particular, three containers are **not** three microservices : the modular monolith decision stands. What is containerized is one application, its database and its static server.

7.2.4.2 Image construction

Both images are built in two stages, so that the build toolchain never reaches the runtime image : the backend image contains a Java runtime and the application archive, with neither Maven nor sources ; the frontend image contains a web server and the compiled assets, with no Node runtime. The backend container runs as a dedicated unprivileged user rather than as root.

Tests are deliberately **not** executed during image construction. Building an image is a packaging step ; the quality gate belongs to the pipeline. Running the suite in both places would double build time without producing any additional signal.

7.2.4.3 Single origin through the reverse proxy

The web server serves the application and forwards `/api` to the backend container. The browser therefore contacts a single origin, which has a concrete consequence : the cross-origin configuration required by the development setup becomes unnecessary, and the refresh cookie, restricted to its own path and marked `SameSite=Strict`, works without any exception. The containerized deployment is, on this point, simpler and safer than the development loop.

7.2.4.4 Startup ordering and health probes

Composing three services raises an ordering problem : a container being *started* is not the same as a container being *ready*. The stack therefore relies on health probes rather than on declaration order. The database is probed over TCP, because the temporary server used during initialisation accepts only local socket connections and would otherwise be reported ready too early. The backend exposes a health endpoint, deliberately left unauthenticated but returning no detail, and is given a generous startup window because its first run applies the full set of schema migrations, including the demonstration dataset.

7.2.4.5 Defects revealed by the exercise

Preparing the pipeline exposed three pre-existing defects that native execution had concealed. The most consequential concerned the frontend build configuration : the production configuration never substituted the production environment file, so the production bundle embedded the development API address. On the development machine the application appeared to work, because the bundle was calling the local backend, while it would have been broken on any other host. The second was a build wrapper documented in the project files but absent from the repository. The third was an ignore rule that silently excluded the configuration template from version control, precisely the file the deployment procedure requires.

That these defects surfaced only when the delivery chain was automated is itself the argument for automating it.

7.2.4.6 Pipeline

The pipeline separates what must always hold from what is only meaningful on integration branches. Compilation, the backend test suite, the production frontend build and the construction of both images run on every proposed change : a component that only breaks after merging is not a gate. The end-to-end test, which starts the full stack and performs a real authentication through the reverse proxy, runs on integration branches, where its cost is justified. Image publication is reserved for the main branch.

The frontend job also carries an explicit guard against the defect described above : it fails if the development address reappears in the production bundle.

7.2.5 Tests and Validation

Test case	Steps	Expected result
Environment substitution	Inspect the production bundle	The development address is absent
Unprivileged execution	Inspect the identity inside the backend container	Non-root user
Full stack startup	Start the three services	All three reported healthy
Schema migration	Count the applied migrations on a pristine volume	All migrations applied
End-to-end authentication	Authenticate through the reverse proxy	Token returned and refresh cookie set
Client-side routing	Reload a deep application route	The application is served, not a 404
Persistence	Stop and restart the stack	Data preserved ; a volume reset replays the migrations
Development loop	Run the native backend and frontend while the stack is up	Both coexist, no port collision

TABLEAU 7.3 : Test cases — Sprint 9

7.2.6 Sprint Outcome

The platform is deployable by a single command on any machine running a container engine, and every change is now verified automatically. The deployment procedure, the environment variables, the backup method and the known pitfalls are documented.

7.2.7 Final State of the Product

At the end of the nine sprints, PMS covers the whole scope identified during the analysis of the Excel workbook : dynamic authorization, project and team management, workload planning and recording, billing, missions, governance and the earned-value indicator engine, all behind a unified interface. Table 7.4 summarises the path from the reference workbook to the delivered platform.

Sprint	Increment delivered	Excel logic replaced
Sprint 1	Technical foundation, authentication	— (no spreadsheet equivalent)

Sprint	Increment delivered	Excel logic replaced
Sprint 2	Dynamic authorization, users, resources, cost rates	TCC sheet
Sprint 3	Projects and lifecycle	Identification sheet
Sprint 4	Teams, workload plan and actuals	Forecast cost and actual cost sheets
Sprint 5	Billing, payments, amendments, missions	Billing progress and missions sheets
Sprint 6	Governance registers	Risks, deliverables, stakeholders, changes sheets
Sprint 7	Internal quote and indicator engine	Statistics and dashboard sheets
Sprint 8	Interface consolidation, tests, security review	— (quality)
Sprint 9	Containerization and continuous delivery	— (delivery chain)

TABLEAU 7.4 : From the reference workbook to the delivered platform

Conclusion

Release 4 closed the construction of the platform by addressing what functionality alone does not deliver : a coherent interface, a verified security model and a reproducible deployment. The incremental path followed since Release 1 (foundation, operations, finance, consolidation) produced a platform whose every increment was demonstrable in isolation, and whose final state covers the entire business logic previously carried by the Excel workbook.

General Conclusion and Perspectives

The objective of this final year project was to replace a scattered, spreadsheet-based project steering process with a centralised platform that preserves the richness of the existing financial model while removing its fragility. That objective is met : PMS covers the whole functional scope identified during the analysis of the reference workbook, from dynamic authorization to the earned-value indicator engine.

Assessment

The construction proceeded through nine increments, each demonstrable in isolation. Release 1 delivered the technical foundation, authentication and above all the dynamic authorization model. Release 2 opened operational steering with projects, teams and workload. Release 3 delivered the core business value : billing, missions, governance and the indicator engine. Release 4 consolidated the interface, verified the security model, and made the platform reproducibly deployable through containerization and a continuous integration pipeline.

Three design decisions structure the result and are, in retrospect, the ones that mattered most.

The first is the **dynamic authorization model**. By inserting a permission level between roles and modules, and by keeping the role–permission matrix in the database, the platform makes its security policy configurable data rather than compiled code. Granting a new right is an administrative act, not a release.

The second is the **hybrid indicator engine**. Rather than choosing between recomputing indicators on demand and persisting them, the platform does both : live computation guarantees that no dashboard displays a stale figure, while a frozen monthly snapshot preserves the official state of each review. This reproduces the strength of the original workbook, a live dashboard with monthly history, without its fragility.

The third is the **normalisation of the monthly time series**. Replacing a grid of month columns with one row per project, resource and month turned a structural limitation of the spreadsheet into a simple insertion, and made integrity constraints expressible : no duplicate month, no entry on an unassigned project, no silent rewriting of a validated period.

Skills Acquired

Beyond the technologies used, this project required reading a real business model out of an artefact that was never written as a specification, and arbitrating between what the workbook did,

what the written specifications said, and what the code could reasonably guarantee. It also required designing for confidentiality : deciding that commercially sensitive amounts should be computed on read rather than stored is an engineering decision with a business justification, not a technical preference.

Limitations and Perspectives

Several avenues remain open.

Automatic time-tracking import. The workload model was designed to accept a second data source alongside manual entry, so that effort recorded in an external time-tracking tool could be imported and reconciled. The mechanism is anticipated in the design but not implemented ; it is the most immediate extension.

Reporting and export. The platform computes and displays the indicators ; generating exportable review documents in the format the company already circulates would shorten the last manual step of the monthly review.

Integration with the information system. The business rules explicitly anticipate future integration points with accounting, human resources and enterprise systems. The modular organisation of the code was chosen with that reversibility in mind : a module can be extracted behind an interface without restructuring the whole.

Scaling the deployment. The current deployment targets the internal use of one company. A significant growth in the number of projects or users would justify revisiting the modular monolith, a decision deliberately taken as reversible rather than permanent.

The platform delivered is therefore both an answer to an immediate operational need and a foundation on which the company's project steering can continue to evolve.

Webographie

- [1] « Spring Boot Reference Documentation. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-boot/documentation.html>
- [2] « Spring Security Reference — Authentication and Authorization. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-security/reference/>
- [3] « Spring Data JPA Reference Documentation. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-data/jpa/reference/>
- [4] « Spring Boot Actuator — Production-ready Features. » [Accessed August 2026]. adresse : <https://docs.spring.io/spring-boot/reference/actuator/index.html>
- [5] « MapStruct Reference Guide — Java Bean Mappings. » [Accessed August 2026]. adresse : <https://mapstruct.org/documentation/stable/reference/html/>
- [6] « PostgreSQL 17 Documentation. » [Accessed August 2026]. adresse : <https://www.postgresql.org/docs/17/index.html>
- [7] « Flyway Documentation — Database Migrations as Code. » [Accessed August 2026]. adresse : <https://documentation.red-gate.com/flyway>
- [8] « Angular Documentation. » [Accessed August 2026]. adresse : <https://angular.dev/overview>
- [9] « Angular — Signals and Reactive State. » [Accessed August 2026]. adresse : <https://angular.dev/guide/signals>
- [10] « Transloco — Internationalization Library for Angular. » [Accessed August 2026]. adresse : <https://jsverse.gitbook.io/transloco>
- [11] « Bootstrap 5 Documentation. » [Accessed August 2026]. adresse : <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
- [12] « Chart.js Documentation. » [Accessed August 2026]. adresse : <https://www.chartjs.org/docs/latest/>
- [13] « RFC 7519 — JSON Web Token (JWT). » [Accessed August 2026]. adresse : <https://www.rfc-editor.org/rfc/rfc7519>
- [14] « OWASP Top 10 — Web Application Security Risks. » [Accessed August 2026]. adresse : <https://owasp.org/www-project-top-ten/>
- [15] « OWASP Cheat Sheet Series — Authentication and Session Management. » [Accessed August 2026]. adresse : <https://cheatsheetseries.owasp.org/>

- [16] « Docker — Dockerfile Reference and Multi-stage Builds. » [Accessed August 2026]. adresse : <https://docs.docker.com/reference/dockerfile/>
- [17] « Docker Compose — Compose File Specification. » [Accessed August 2026]. adresse : <https://docs.docker.com/reference/compose-file/>
- [18] « nginx Documentation — HTTP Proxy Module. » [Accessed August 2026]. adresse : https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- [19] « GitHub Actions Documentation — Workflow Syntax. » [Accessed August 2026]. adresse : <https://docs.github.com/en/actions>
- [20] « Apache Maven — Introduction to the Build Lifecycle. » [Accessed August 2026]. adresse : <https://maven.apache.org/guides/introduction/introduction-to-the-lifecycle.html>
- [21] « OMG Unified Modeling Language (UML) Specification, version 2.5.1. » [Accessed August 2026].
adresse : <https://www.omg.org/spec/UML/2.5.1/>
- [22] « PlantUML Language Reference Guide. » [Accessed August 2026]. adresse : <https://plantuml.com/guide>
- [23] « The Scrum Guide — The Definitive Guide to Scrum. » [Accessed August 2026]. adresse : <https://scrumguides.org/scrum-guide.html>
- [24] « Project Management Institute — Earned Value Management. » [Accessed August 2026]. adresse : <https://www.pmi.org/learning/library/earned-value-management-systems-analysis-8026>

Résumé

FR

Mots clés : KW1, KW2

Abstract

EN

Keywords : KW1, KW2